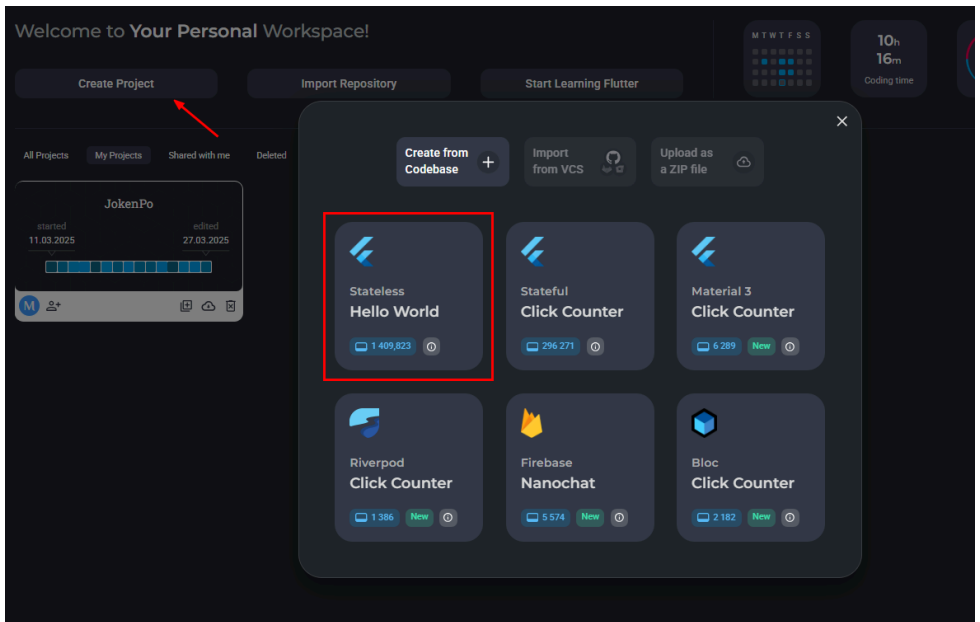


Criando uma Pokédex com Firebase Auth e API

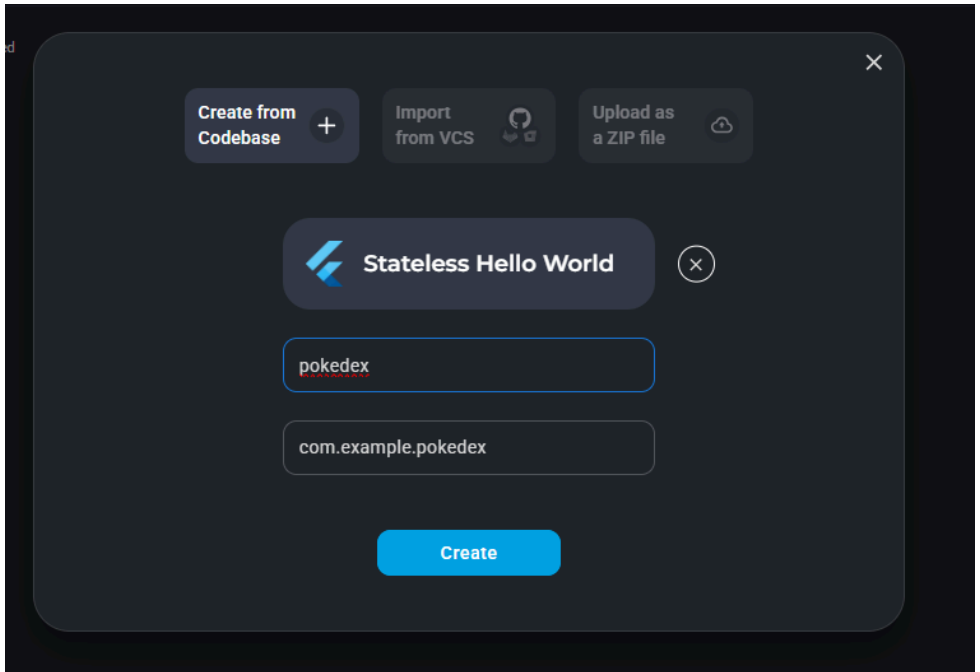
Iniciando o projeto.....	2
Criando o Login.....	7
Adicionando Imagens.....	11
Instalando Pacotes com Pub.dev.....	13
Importando Fonts.....	18
Criando a Tela de Login.....	19
Criando a Tela de Cadastro.....	27
Configurando o Firebase.....	32
Cadastrando um usuário no Firebase.....	57
Feedback com SnackBar.....	60
Criando a tela da Pokédex.....	64
Realizando login via Firebase.....	79
Código-fonte.....	86

Iniciando o projeto

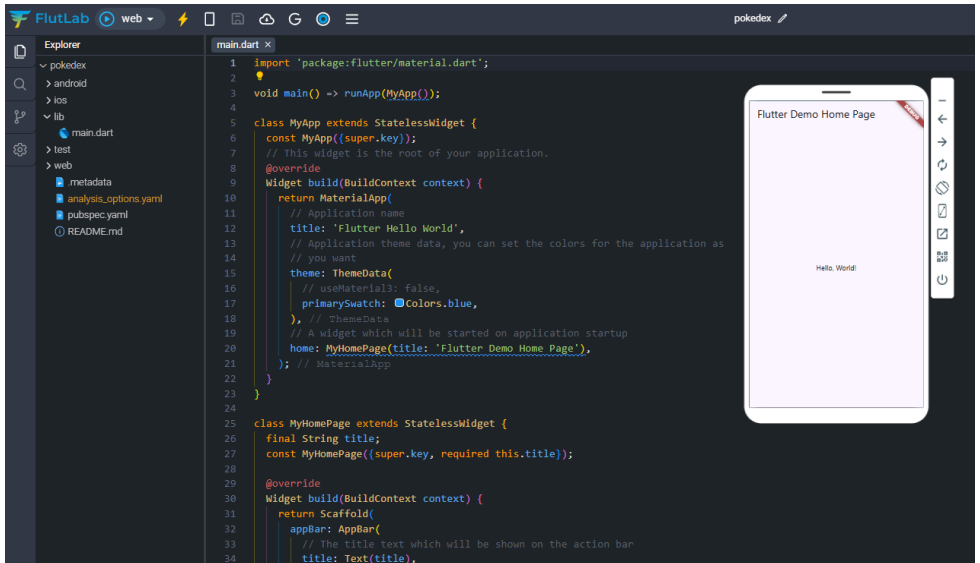
1 - Vamos iniciar um novo projeto no <https://flutlab.io/workspace>, após clicar em Create Project selecione a opção Stateless Hello World:



2 - Coloque o nome de pokedex e depois create:



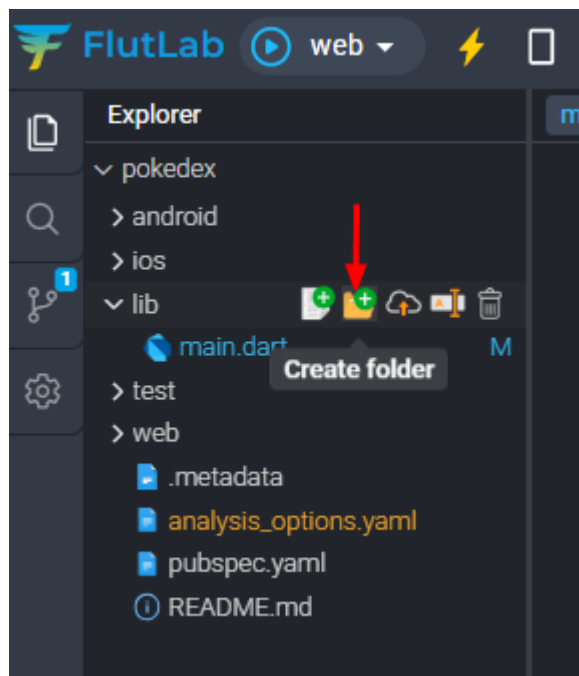
3 - Após abrir seu projeto ele terá essa aparência



4 - Vamos retirar o código referente ao template para iniciar o app do zero, veja que nada aparece no simulador pois de fato em nosso código não estamos passando nenhuma tela para ser exibida.

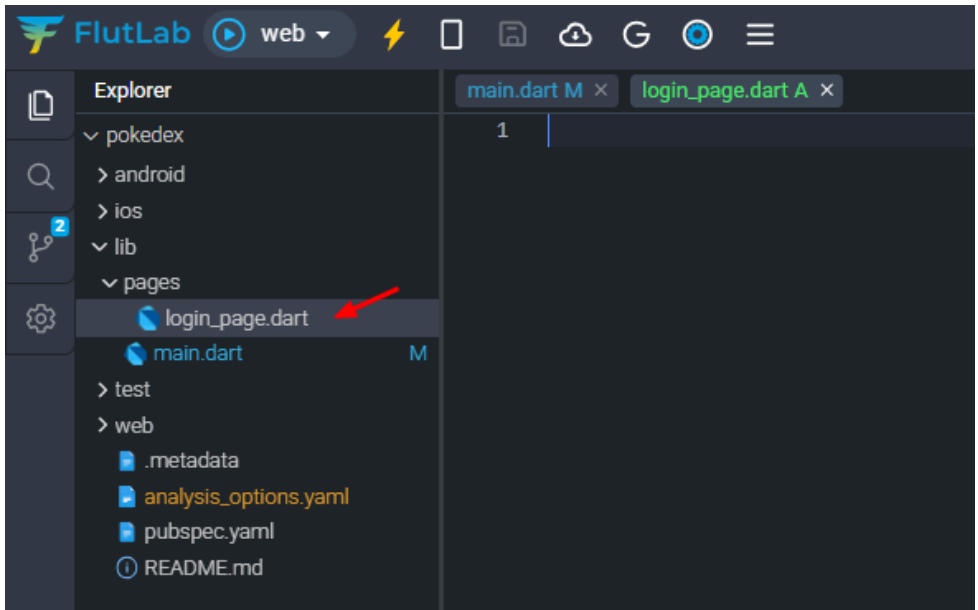
```
main.dart M x
1  import 'package:flutter/material.dart';
2
3  void main() => runApp(MyApp());
4
5  class MyApp extends StatelessWidget {
6      const MyApp({super.key});
7
8      @override
9      Widget build(BuildContext context) {
10         return MaterialApp(
11             debugShowCheckedModeBanner: false,
12         );
13     }
14 }
15
```

5 - Visando boas práticas, vamos criar uma nova pasta dentro de lib para organizar nossas telas. Chame a pasta **pages**:



Criando o Login

6 - Dentro da pasta pages crie um novo arquivo chamado login_page.dart:



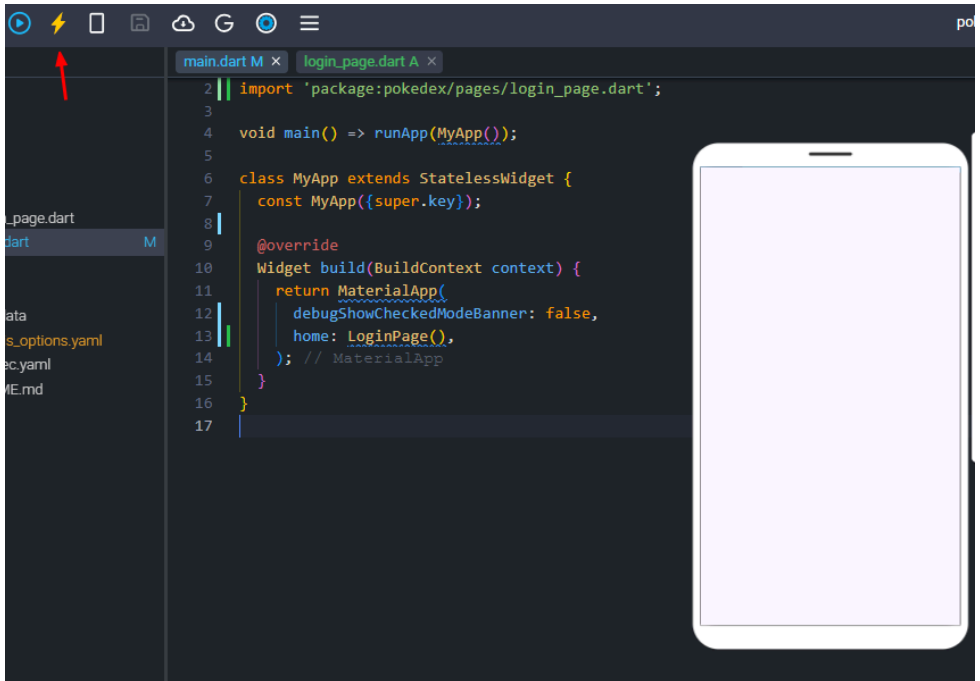
7 - Vamos criar a estrutura básica de uma tela em Flutter, chamaremos a classe dessa tela de LoginPage:

```
main.dart M x login_page.dart A x
1  import 'package:flutter/material.dart';
2
3  class LoginPage extends StatefulWidget {
4      const LoginPage({super.key});
5
6      @override
7      State<LoginPage> createState() => _LoginPageState();
8  }
9
10 class _LoginPageState extends State<LoginPage> {
11     @override
12     Widget build(BuildContext context) {
13         return Scaffold();
14     }
15 }
16
```


8 - Para que a tela de login seja exibida quando o app iniciar vamos voltar ao arquivo `main.dart` e dentro do `MaterialApp` vamos adicionar a propriedade `home` e chamar a classe `LoginPage()` que criamos anteriormente:

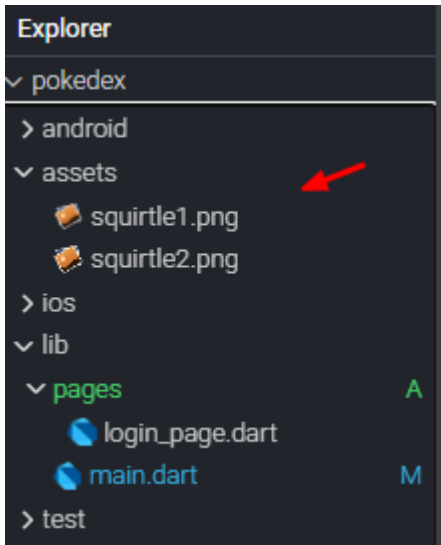
```
2 || import 'package:pokedex/pages/login_page.dart';
3
4 void main() => runApp(MyApp());
5
6 class MyApp extends StatelessWidget {
7   const MyApp({super.key});
8
9   @override
10  Widget build(BuildContext context) {
11    return MaterialApp(
12      debugShowCheckedModeBanner: false,
13      home: LoginPage(),
14    ); // MaterialApp
15  }
16 }
17
```

9 - Faça o Hot Reload (ou build), veja que agora a aplicação aparece a tela do LoginPage que neste momento é apenas uma tela em branco:

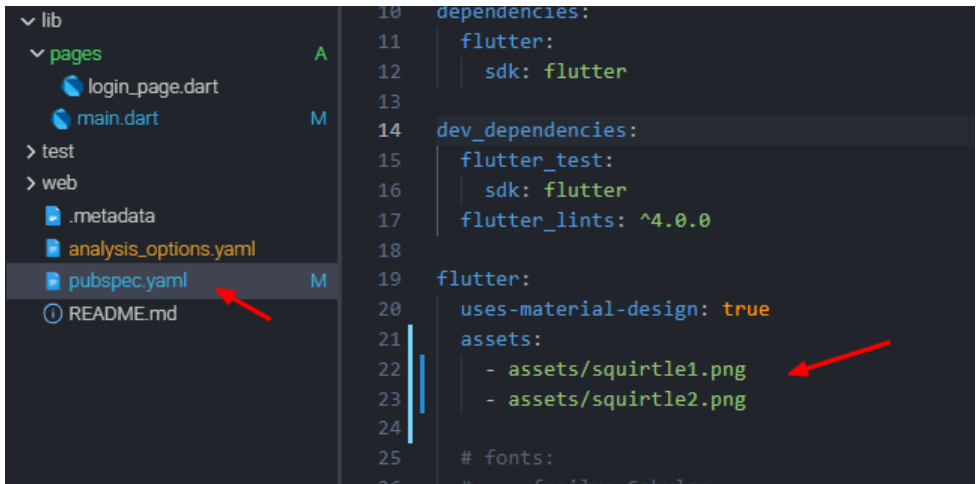


Adicionando Imagens

10 - Crie uma nova pasta chamada assets na raiz do projeto, importe as duas imagens abaixo (AVA) como abaixo:

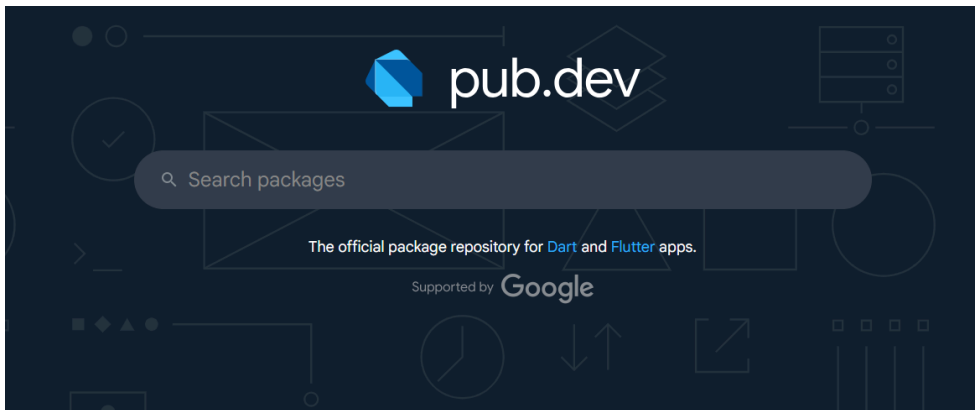


11 - Abra o arquivo pubspec.yaml e vamos colocar as imagens como assets:

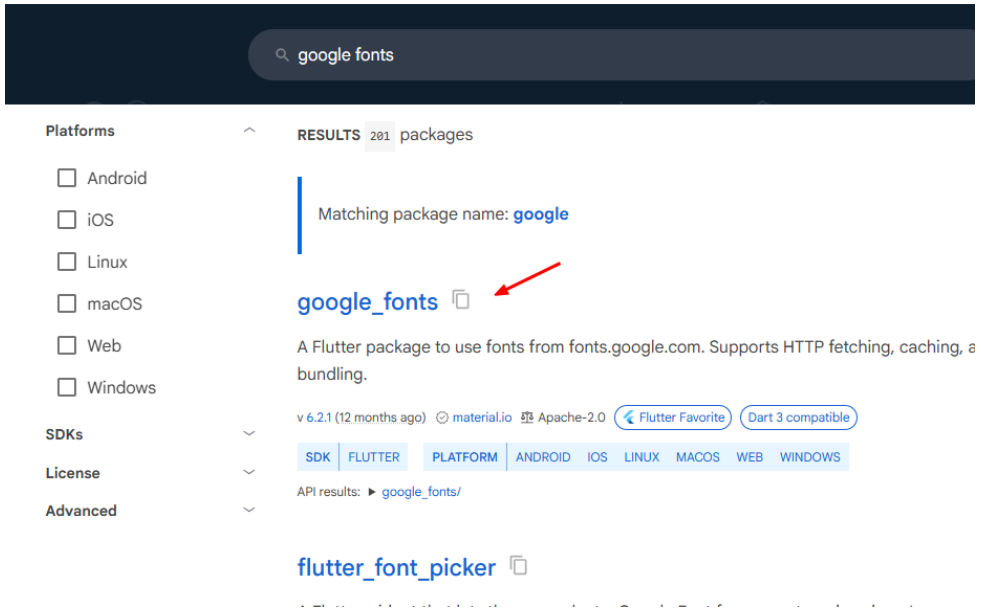


Instalando Pacotes com Pub.dev

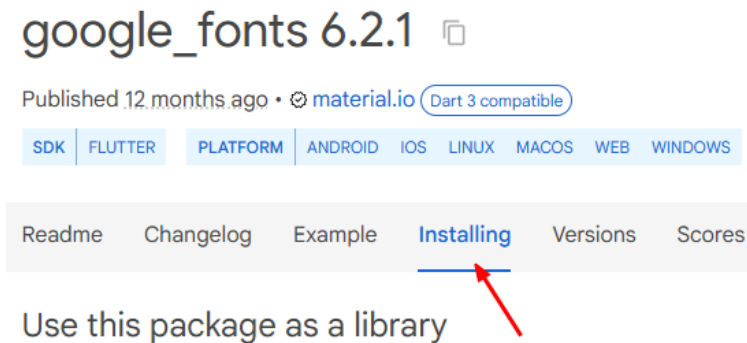
12 - Vamos aproveitar também e trabalhar com fontes customizadas. Sempre que precisar instalar algum package novo visite <https://pub.dev/>, esse é o repositório oficial de packages do Dart e Flutter:



13 - Procure o package google fonts e o abra:



14 - Além de exemplos de uso podemos clicar em Installing e seguir os passos de instalação



Veja que na documentação temos que alizar uma instalação com o seguinte comando:

Use this package as a library

Depend on it

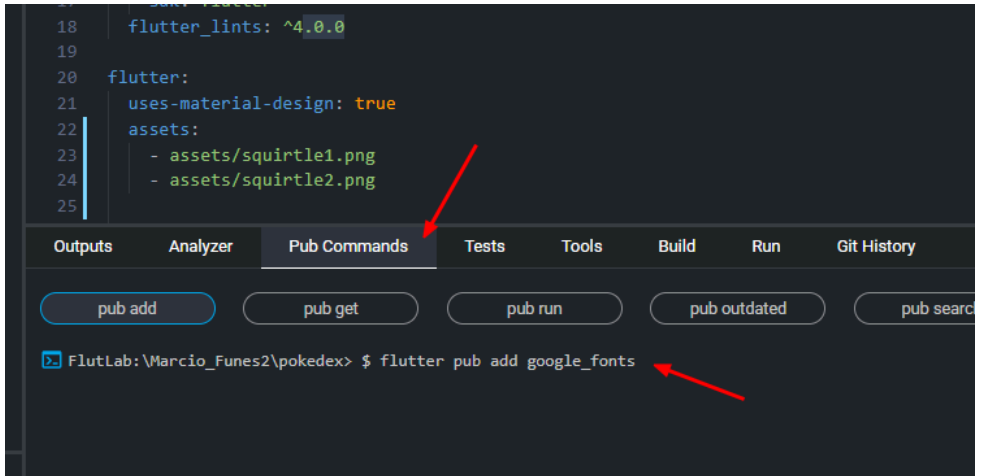
Run this command:

With Flutter:

```
$ flutter pub add google_fonts
```



15 - Na aba Pub Commands execute o comando: **flutter pub add google_fonts**



16 - Veja que o package foi adicionado com sucesso e no pubspec.yaml foi adicionado a versão dessa nova dependência instalada:

```
9
10 dependencies:
11   flutter:
12     sdk: flutter
13   google_fonts: ^6.2.1
14
15 dev_dependencies:
16   flutter_test:
17     sdk: flutter
18   flutter_lints: ^4.0.0
19
20 flutter:
21   uses-material-design: true
22   assets:
23     - assets/squirtle1.png
24     - assets/squirtle2.png
25
```

Outputs Analyzer Pub Commands Tests Tools Build Run Git Hi

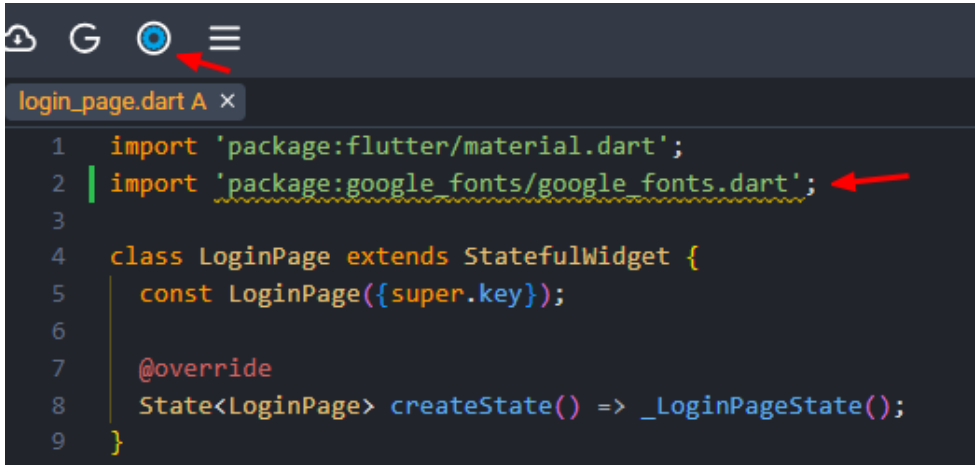
pub add pub get pub run pub outdated

Running "flutter pub add" in FlutLab\Marcio_Funes2\pokedex>

✓ Package added successfully. Run "pub get" command

Importando Fonts

17 - Agora podemos no arquivo login_page.dart importar o google fonts e usar em nosso desenvolvimento. Caso essa linha apresente algum erro, clique em Restart Analyser para correção:



The screenshot shows an IDE window titled 'login_page.dart A x'. The code is as follows:

```
1 import 'package:flutter/material.dart';
2 import 'package:google_fonts/google_fonts.dart';
3
4 class LoginPage extends StatefulWidget {
5   const LoginPage({super.key});
6
7   @override
8   State<LoginPage> createState() => _LoginPageState();
9 }
```

Two red arrows point to the import statements: one to the Flutter import on line 1 and another to the Google Fonts import on line 2. The Google Fonts import line has a yellow wavy underline, indicating a warning or error.

Criando a Tela de Login

18 - Vamos agora customizar nossa tela de login, vamos usar essa tela como referência:

 **Pokedex**

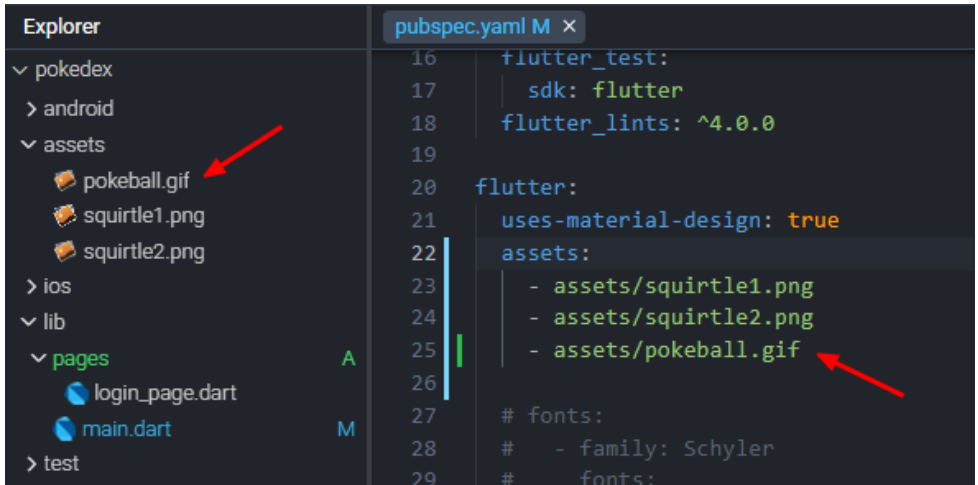


Login

Login

Não tem um cadastro? [Cadastre-se](#)

19 - Adicione um gif aos assets (AVA) e também no pubspec.yaml



20 - Nas linhas 1 ao 20 temos a estrutura inicial do app, as caixas de texto para email e senha que por hora apenas vão printar no console:

```
login_page.dart x
1  import 'package:flutter/material.dart';
2  import 'package:google_fonts/google_fonts.dart';
3
4  class LoginPage extends StatefulWidget {
5      const LoginPage({super.key});
6
7      @override
8      State<LoginPage> createState() => _LoginPageState();
9  }
10
11 class _LoginPageState extends State<LoginPage> {
12     final TextEditingController _emailController = TextEditingController();
13     final TextEditingController _passwordController = TextEditingController();
14
15     void _login() {
16         String email = _emailController.text;
17         String password = _passwordController.text;
18         print('Email: $email');
19         print('Password: $password');
20     }
21 }
```

21 - Nas linhas 22 a 45 temos a adição do Scaffold que organiza o appBar (barra no topo) com a imagem da pokebola, o título Pokedex e a cor de fundo vermelha.

```
21
22  @override
23  Widget build(BuildContext context) {
24    return Scaffold(
25      appBar: AppBar(
26        title: Row(
27          children: [
28            Image.asset(
29              'assets/pokeball.gif',
30              height: 40,
31              width: 40,
32            ), // Image.asset
33            const SizedBox(width: 8),
34            Text(
35              'Pokedex',
36              style: GoogleFonts.limelight(
37                fontSize: 24,
38                fontWeight: FontWeight.bold,
39              ),
40          ), // Text
41        ],
42      ), // Row
43      backgroundColor: Colors.red,
44      elevation: 0,
45    ), // AppBar
```

22 - Das linhas 46 até 72 temos a definição da cor do fundo (linha 47), a adição da imagem centralizada, o texto login. Veja que estamos usando o recurso GoogleFonts. o nome da fonte que desejar:

Consulte as fontes disponíveis: <https://fonts.google.com/>

```
44     elevation: 0,
45   ), // AppBar
46   body: Container(
47     color: Color(0xffffffff),
48     child: Padding(
49       padding: const EdgeInsets.all(16.0),
50       child: Column(
51         mainAxisAlignment: MainAxisAlignment.center,
52         children: <Widget>[
53           Center(
54             child: Image.asset(
55               'assets/squirtle1.png',
56               height: 200,
57               width: 200,
58             ), // Image.asset
59           ), // Center
60           const SizedBox(height: 32),
61           Align(
62             alignment: Alignment.centerLeft,
63             child: Text(
64               'Login',
65               style: GoogleFonts.dmSerifText(
66                 fontSize: 28,
67                 fontWeight: FontWeight.bold,
68                 color: Color(0xff000000),
69               ),
70             ), // Text
71           ), // Align
72           const SizedBox(height: 32),
```

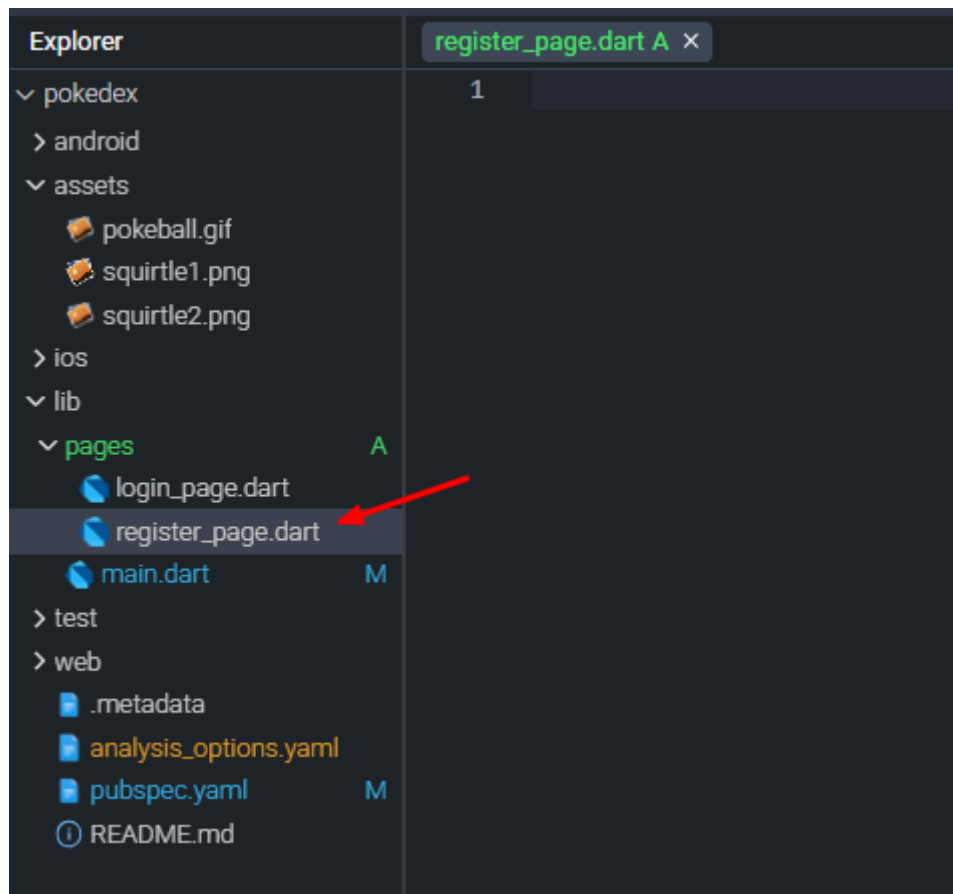
23 - Nas linhas 73 até 102 temos o texto que irá dentro da caixa de texto Email e Password além de colocarmos o botão de cadastro na cor vermelha:

```
70     ), // Text
71   ), // Align
72   const SizedBox(height: 32),
73   TextField(
74     controller: _emailController,
75     decoration: const InputDecoration(
76       labelText: 'Email',
77       labelStyle: TextStyle(color: Colors.grey),
78       border: OutlineInputBorder(),
79     ), // InputDecoration
80     keyboardType: TextInputType.emailAddress,
81   ), // TextField
82   const SizedBox(height: 16),
83   TextField(
84     controller: _passwordController,
85     decoration: const InputDecoration(
86       labelText: 'Password',
87       labelStyle: TextStyle(color: Colors.grey),
88       border: OutlineInputBorder(),
89     ), // InputDecoration
90     obscureText: true,
91   ), // TextField
92   const SizedBox(height: 16),
93   ElevatedButton(
94     onPressed: _login,
95     child: const Text(
96       'Login',
97       style: TextStyle(color: Colors.black),
98     ), // Text
99     style: ElevatedButton.styleFrom(
100       backgroundColor: Colors.red,
101     ),
102   ), // ElevatedButton
```


24 - Na linha 103 a 123 temos o texto de 'Não tem um cadastro?' e um link para levar o usuário a outra tela:

```
100         backgroundColor: Colors.red,
101     ),
102 ), // ElevatedButton
103 const SizedBox(height: 32),
104 Row(
105     mainAxisAlignment: MainAxisAlignment.center,
106     children: [
107         const Text('Não tem um cadastro?'),
108         TextButton(
109             onPressed: () {},
110             child: const Text(
111                 'Cadastre-se',
112                 style: TextStyle(color: Colors.blue),
113             ), // Text
114         ), // TextButton
115     ],
116 ), // Row
117 ], // <Widget>[]
118 ), // Column
119 ), // Padding
120 ), // Container
121 ); // Scaffold
122 }
123 }
124
```

25 - Crie um novo arquivo chamado register_page.dart na pasta pages:



Criando a Tela de Cadastro

26 - Copie todo o código do arquivo login_page.dart para o register_page.dart. Essa será a tela de cadastro:





Cadastrar

Cadastro

Pokémon! Temos que pegar

vamos fazer algumas pequenas alterações:

```
register_page.dart A x
1  import 'package:flutter/material.dart';
2  import 'package:google_fonts/google_fonts.dart';
3
4  class RegisterPage extends StatefulWidget {
5      const RegisterPage({super.key});
6
7      @override
8      State<RegisterPage> createState() => _RegisterPageState();
9  }
10
11  class _RegisterPageState extends State<RegisterPage> {
12      final TextEditingController _emailController = TextEditingController();
13      final TextEditingController _passwordController = TextEditingController();
14
15      void _login() {
16          String email = _emailController.text;
17          String password = _passwordController.text;
18          print('Email: $email');
19          print('Password: $password');
20      }
21  }
```

```
21
22 @override
23 Widget build(BuildContext context) {
24   return Scaffold(
25     appBar: AppBar(
26       backgroundColor: Colors.red,
27       elevation: 0,
28     ), // AppBar
29     body: Container(
30       color: Color(0xffffffff),
31       child: Padding(
32         padding: const EdgeInsets.all(16.0),
33         child: Column(
34           mainAxisAlignment: MainAxisAlignment.center,
35           children: <Widget>[
36             Center(
37               child: Image.asset(
38                 'assets/squirtle2.png',
39                 height: 200,
40                 width: 200,
41               ), // Image.asset
42             ), // Center
43             const SizedBox(height: 32),
44             Align(
45               alignment: Alignment.centerLeft,
46               child: Text(
47                 'Cadastrar',
48                 style: GoogleFonts.dmSerifText(
49                   fontSize: 28,
50                   fontWeight: FontWeight.bold,
51                   color: Color(0xff000000),
52                 ),
53               ), // Text
54             ), // Align
```

```
53     ), // Text
54   ), // Align
55   const SizedBox(height: 32),
56   TextField(
57     controller: _emailController,
58     decoration: const InputDecoration(
59       labelText: 'Digite seu email:',
60       labelStyle: TextStyle(color: Colors.grey),
61       border: OutlineInputBorder(),
62     ), // InputDecoration
63     keyboardType: TextInputType.emailAddress,
64   ), // TextField
65   const SizedBox(height: 16),
66   TextField(
67     controller: _passwordController,
68     decoration: const InputDecoration(
69       labelText: 'Digite seu Password',
70       labelStyle: TextStyle(color: Colors.grey),
71       border: OutlineInputBorder(),
72     ), // InputDecoration
73     obscureText: true,
74   ), // TextField
75   const SizedBox(height: 16),
76   ElevatedButton(
77     onPressed: _login,
78     child: const Text(
79       'Cadastro',
80       style: TextStyle(color: Colors.black),
81     ), // Text
82     style: ElevatedButton.styleFrom(
83       backgroundColor: Colors.red,
84     ),
85   ), // ElevatedButton
86   const SizedBox(height: 32),
87   Row(
88     mainAxisAlignment: MainAxisAlignment.center,
89     children: [
90       const Text('Pokémon! Temos que pegar'),
91       const SizedBox(height: 32),
92     ],
```

```
88         mainAxisAlignment: MainAxisAlignment.center,
89         children: [
90             const Text('Pokémon! Temos que pegar'),
91             const SizedBox(height: 32),
92         ],
93     ), // Row
94 ], // <Widget>[]
95 ), // Column
96 ), // Padding
97 ), // Container
98 ); // Scaffold
99 }
100 }
101
```

Configurando o Firebase

27 - Agora que temos as telas de login e cadastro vamos configurar o back-end da nossa aplicação, acesse o <https://firebase.google.com/?hl=pt-br> e crie uma conta ou use sua conta Google.

Faça seu app  o melhor possível com o Firebase e a IA generativa

Crie e execute experiências modernas com tecnologia de IA que os usuários adoram com o Firebase, uma plataforma projetada para ajudar você durante toda a jornada de desenvolvimento de apps. Com o apoio do Google e a confiança de milhões de empresas no mundo todo.

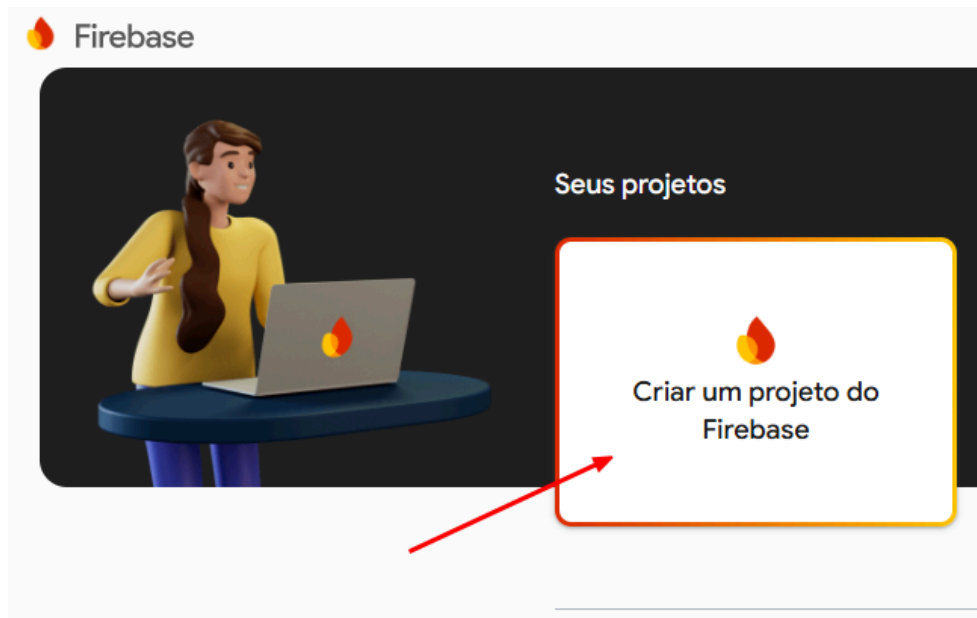
[Começar](#)

[Teste a demonstração →](#)

28 - Após login clique em Go to console:



29 - Clique em Criar um projeto Firebase:



30 - Dê um nome ao seu projeto e clique em continuar:

× Criar um projeto

Vamos começar nomeando o projeto[?]

Nome do projeto

pokedex-app

 pokedex-app-12f28

 unifacef.com.br

Já tem um projeto do Google Cloud?
[Adicionar o Firebase ao projeto do Google Cloud](#)

Continuar

31 - Desabilite o Analytics e clique em Criar projeto:

× Criar um projeto

Google Analytics para seu projeto do Firebase

O Google Analytics é uma solução de análise ilimitada e gratuita. Com ele, é possível segmentar, gerar relatórios e muito mais nos seguintes produtos: Firebase Crashlytics, Cloud Messaging, Mensagens no app, Configuração remota, Teste A/B e Cloud Functions.

O Google Analytics ativa:

× Teste A/B ⓘ

× Gatilhos do Cloud Functions com base ⓘ
em eventos

× Segmentação de usuários em produtos ⓘ
do Firebase

× Geração de relatórios ilimitada gratuita ⓘ

× Registros de navegação estrutural no ⓘ
Crashlytics

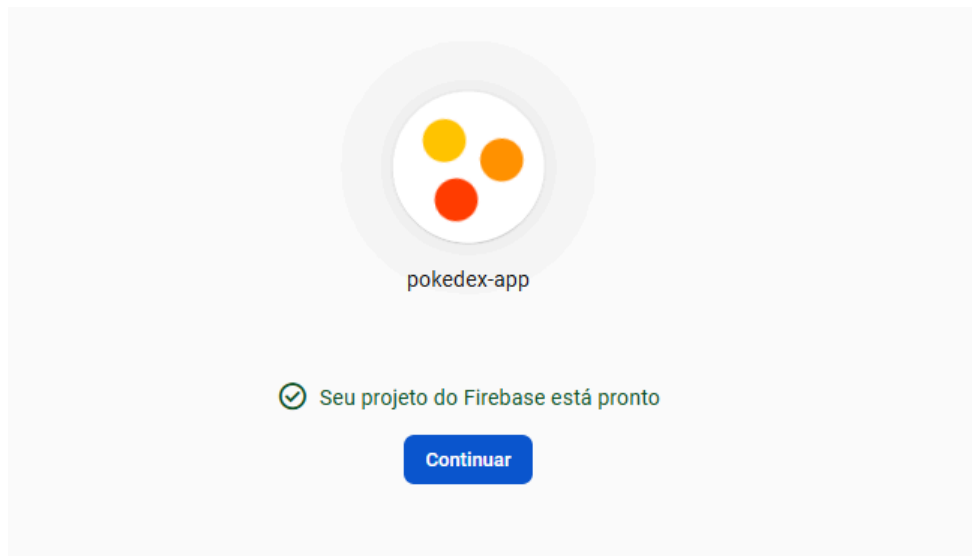


Ativar o Google Analytics neste projeto
Recomendado

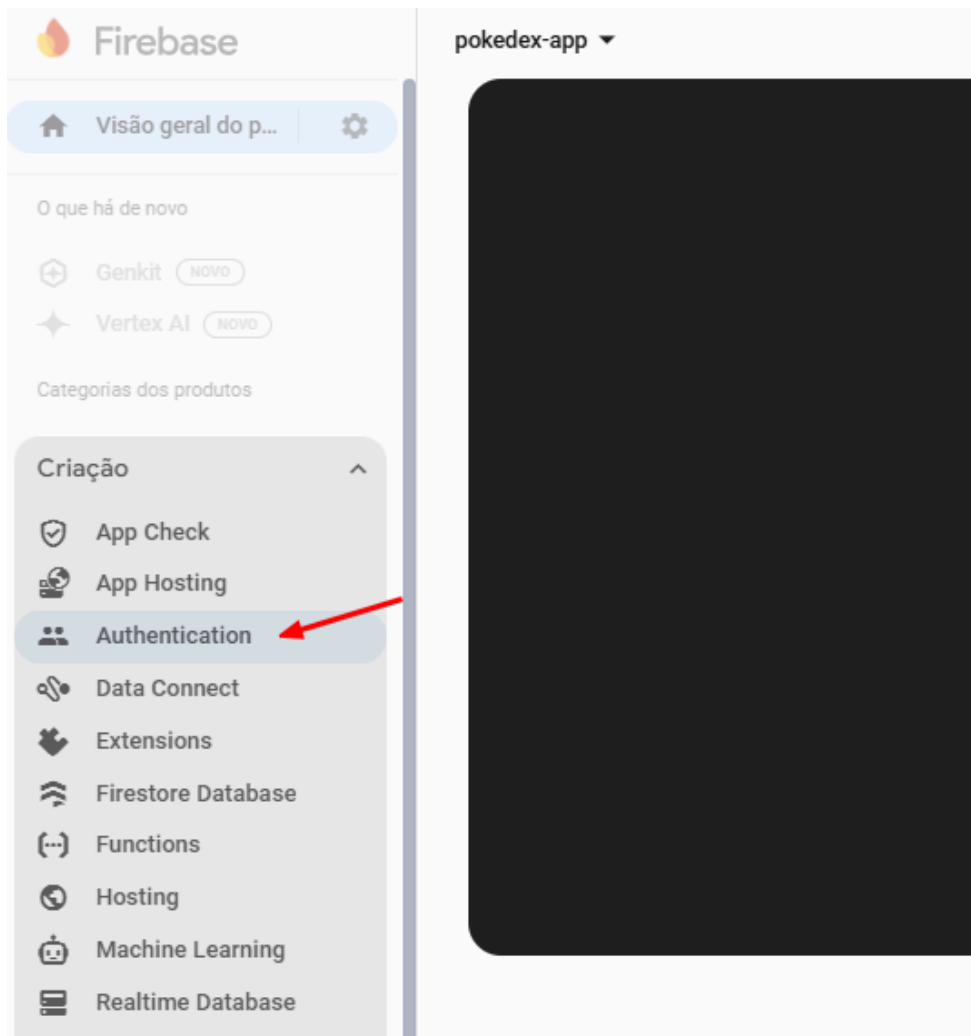
[Anterior](#)

Criar projeto

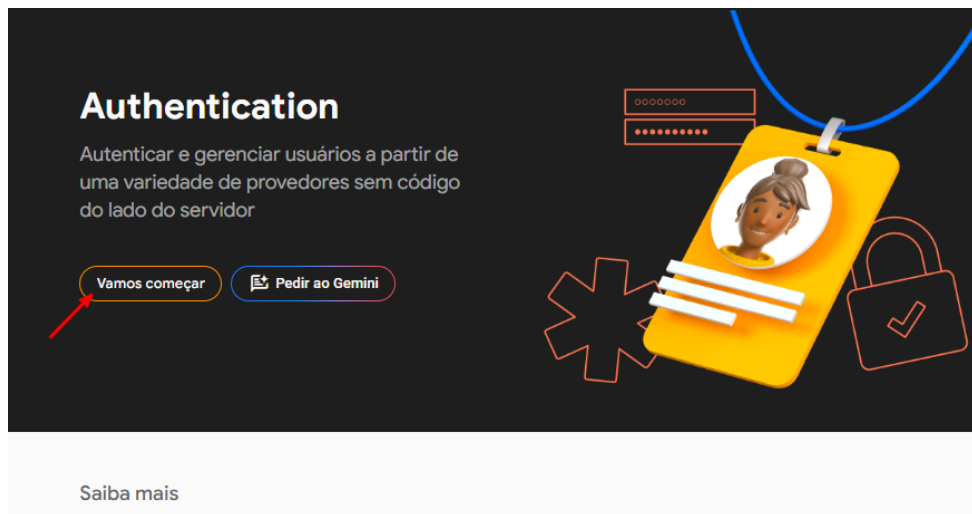
32 - Aguarde a criação do projeto e clique em continuar:



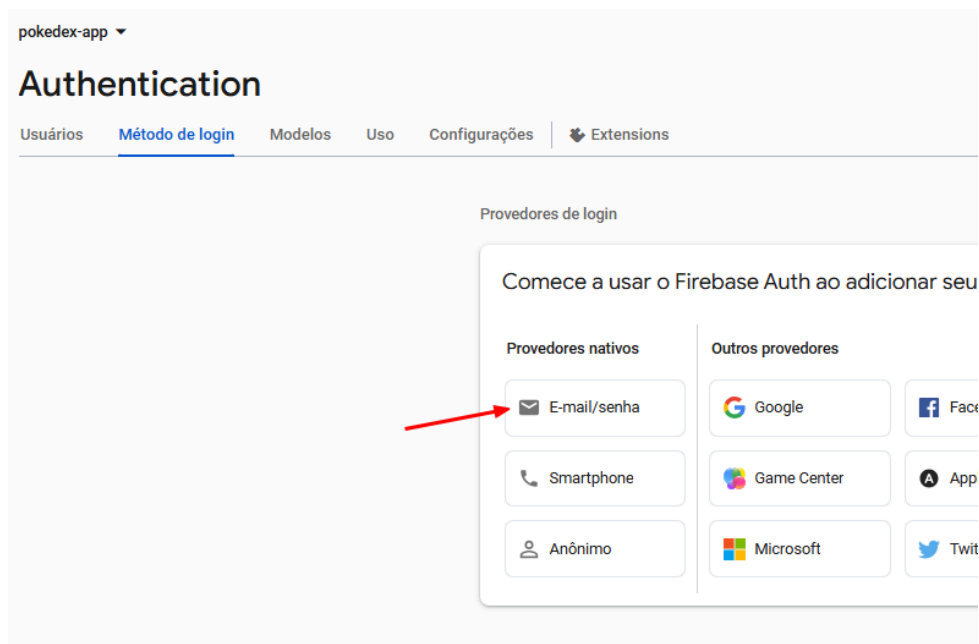
33 - Clique em Authentication:



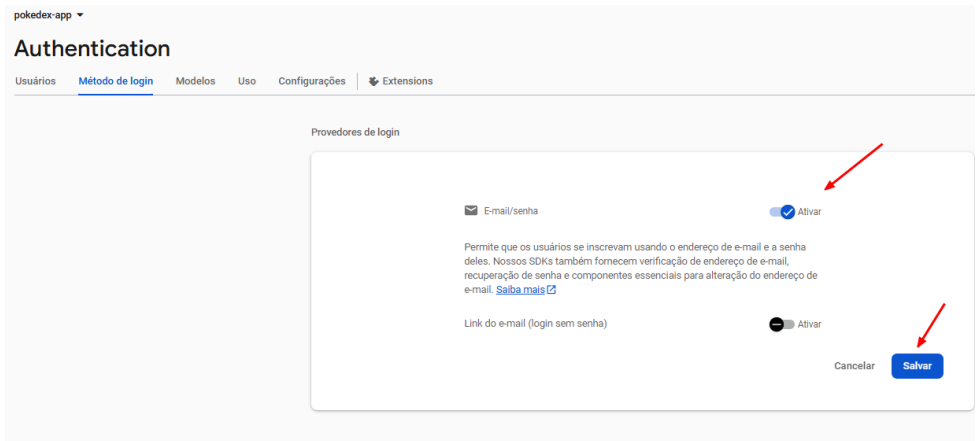
34 - Clique em Vamos começar:



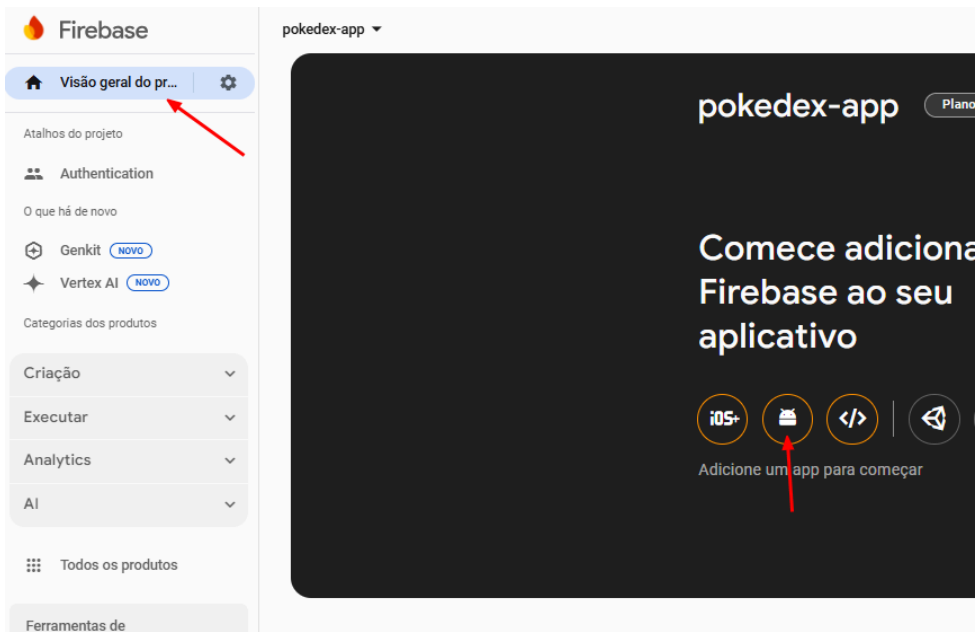
35 - Clique em Email/senha:



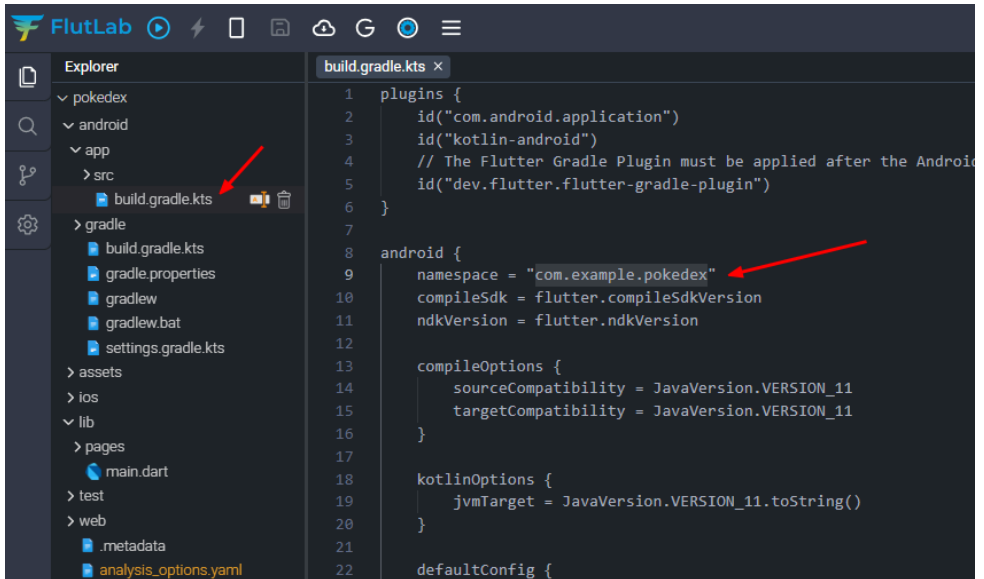
36 - Clique em ativar e depois salvar:



37 - Clique em Visão geral do projeto e depois em Android (ou IOS+ caso use iPhone):



38 - Coloque o nome do pacote do seu aplicativo. Caso tenha esquecido o nome escolhido durante a criação do projeto:



1

②

com.example.pokedex

②

Meu app Android

②

00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00

Registrar app

2

3

4

39 - Dê um apelido ao app e depois Registrar:

✕ Adicionar o Firebase ao seu app Android

1 Registrar app

Nome do pacote do Android (?)

com.example.pokedex

Apelido do app(opcional)

Pokedex App

Certificado de assinatura SHA-1 de depuração (opcional)

00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:00:(

ⓘ Necessário para os Dynamic Links, o Login do Google e o suporte por telefone no Auth. Edite os certificados SHA-1 nas configurações.

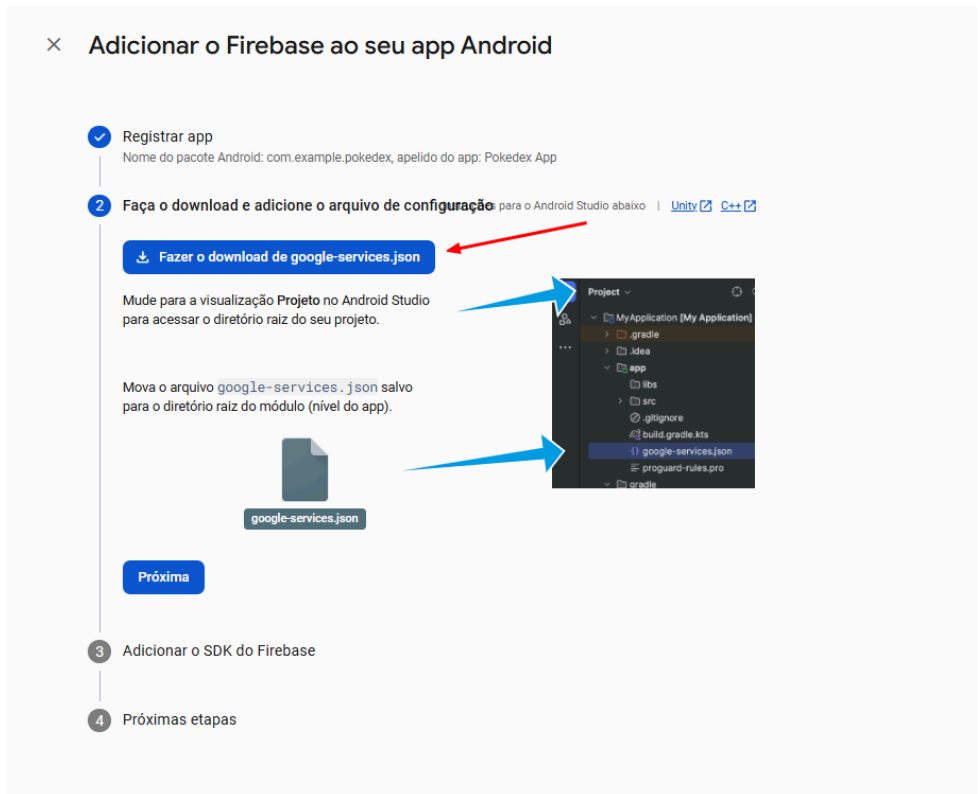
Registrar app

2 Faça o download e adicione o arquivo de configuração

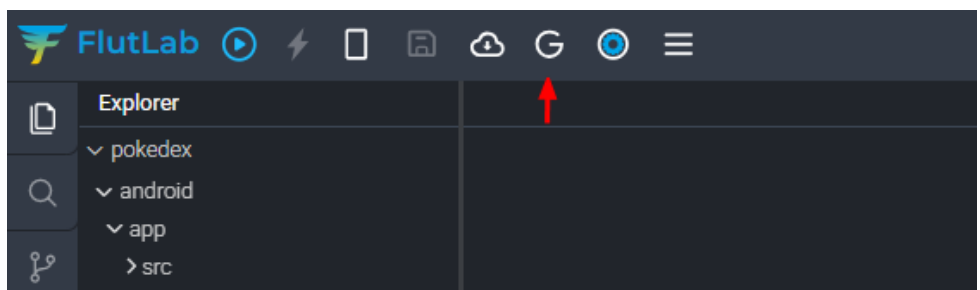
3 Adicionar o SDK do Firebase

4 Próximas etapas

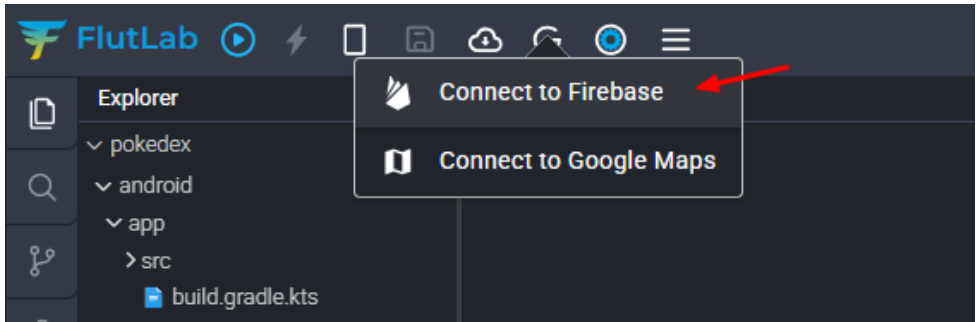
40 - Faça o download do google-service.json, iremos utilizar esse arquivo na sequência, não o perca. Dê próxima até retornar para a tela do projeto:



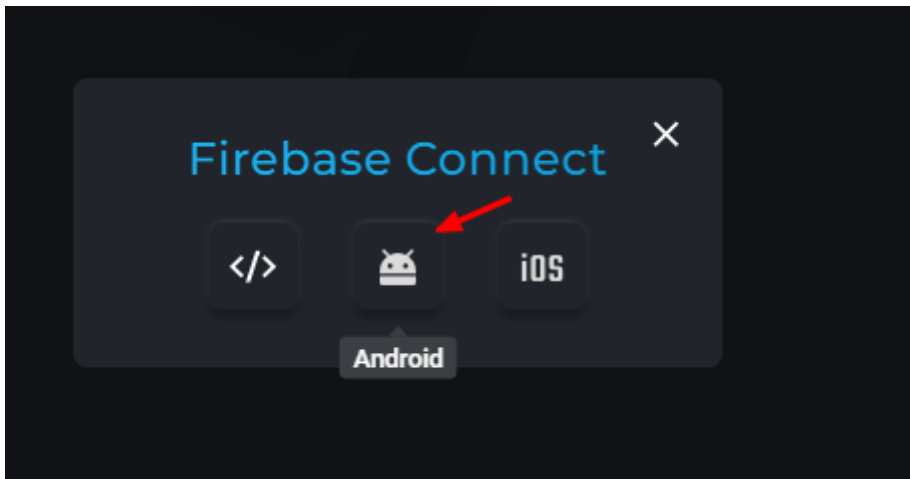
41 - Clique Google Service:



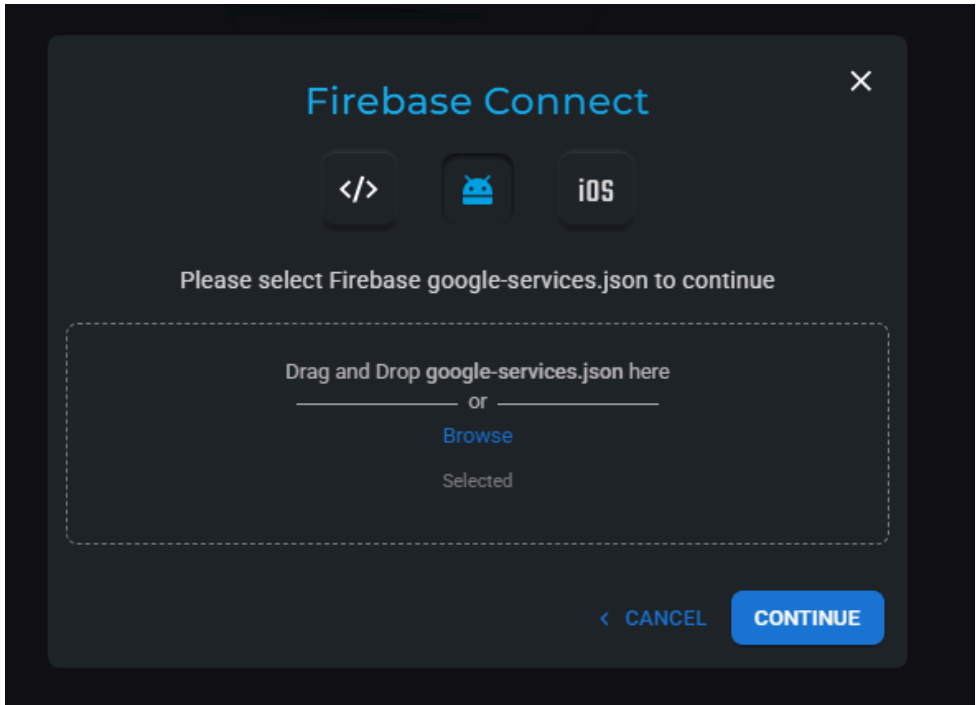
42 - Clique em Connect to Firebase:



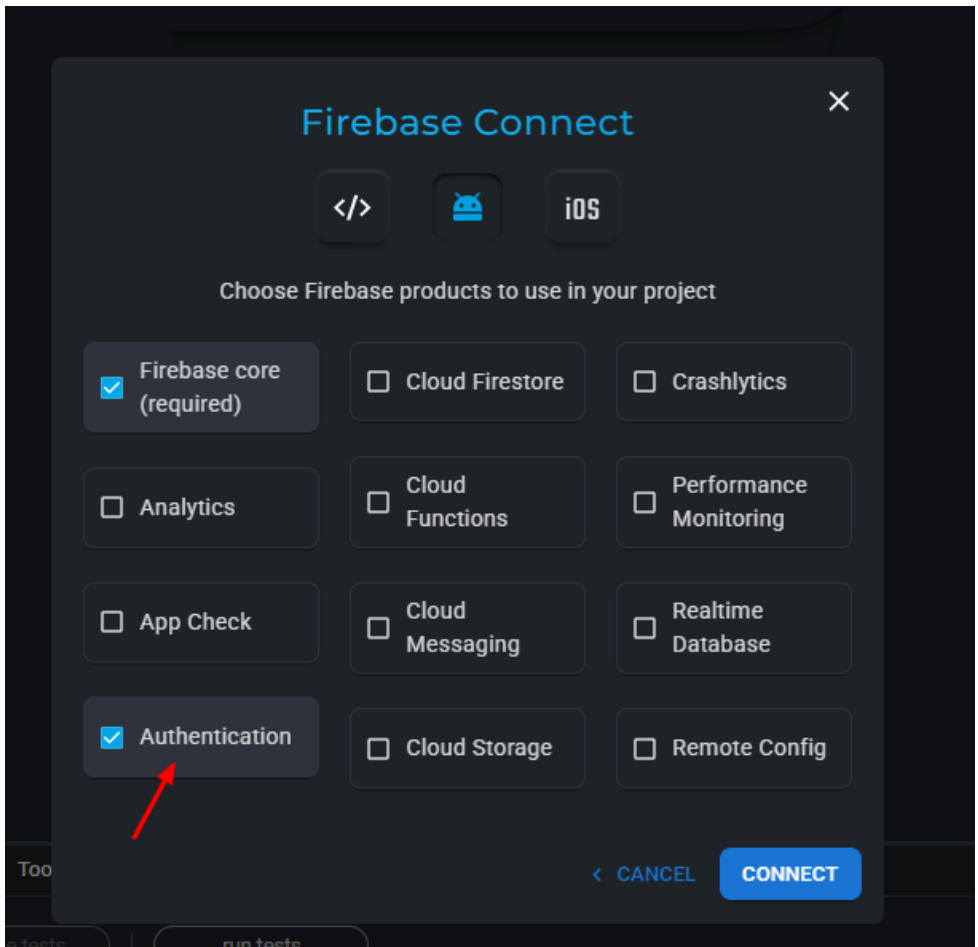
43 - Escolha Android ou IOS:



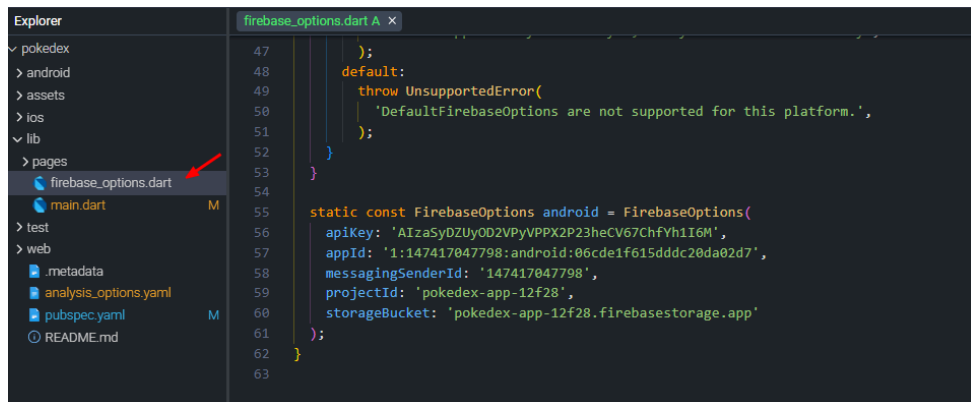
44 - Arraste o arquivo google-services.json que geramos nos passos anteriores. Clique em continue:



45 - Escolha os serviços que deseja integrar, em nosso caso selecione o serviço de Authentication e depois connect:



46 - Veja que após a conexão o projeto criou automaticamente um arquivo de configuração chamado `firebase_options.dart`. Aqui temos informações importantes, inclusive, caso queira no futuro publicar seu app na Play Store:

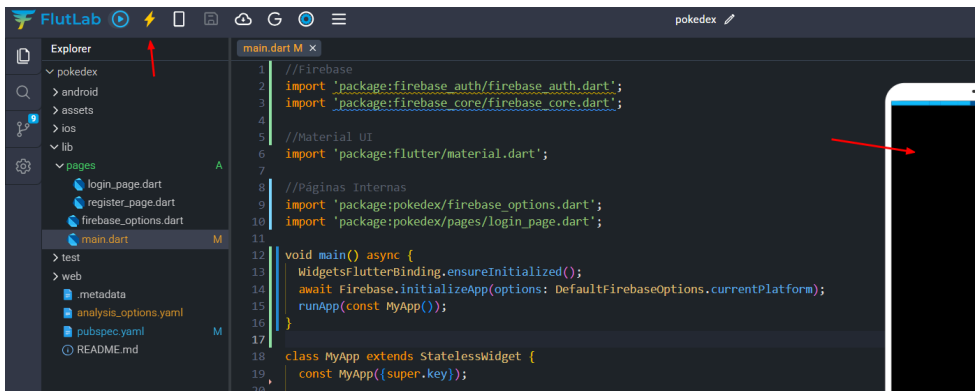


```
47     );
48     default:
49       throw UnsupportedError(
50         'DefaultFirebaseOptions are not supported for this platform.',
51       );
52   }
53 }
54
55 static const FirebaseOptions android = FirebaseOptions(
56   apiKey: 'AIzaSyDZUyOD2VPyVPPX2P23heCV67ChfYh1I6M',
57   appId: '1:147417047798:android:06cde1f615dddc20da02d7',
58   messagingSenderId: '147417047798',
59   projectId: 'pokedex-app-12f28',
60   storageBucket: 'pokedex-app-12f28.firebaseioapp.appspot.com',
61 );
62 }
63
```

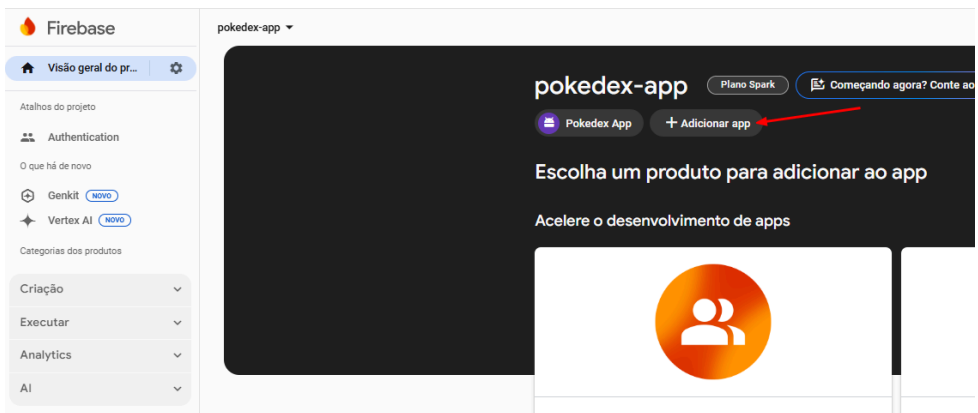
47 - Abra o arquivo main.dart, adicione algumas importações do firebase (linha 2 e 3) e deixe-o como abaixo. Dentro de main estamos inicializando os serviços do Firebase (ensureInitialized) e inicializando o serviço de acordo com a plataforma (Android ou IOS), lembrando que esse código atende as duas plataformas:

```
main.dart M x
1 //Firebase
2 import 'package:firebase_auth/firebase_auth.dart';
3 import 'package:firebase_core/firebase_core.dart';
4
5 //Material UI
6 import 'package:flutter/material.dart';
7
8 //Páginas Internas
9 import 'package:pokedex/firebase_options.dart';
10 import 'package:pokedex/pages/login_page.dart';
11
12 void main() async {
13   WidgetsFlutterBinding.ensureInitialized();
14   await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
15   runApp(const MyApp());
16 }
17
18 class MyApp extends StatelessWidget {
19   const MyApp({super.key});
20
21   @override
22   Widget build(BuildContext context) {
23     return const MaterialApp(
24       debugShowCheckedModeBanner: false,
25       home: LoginPage(),
26     ); // MaterialApp
27   }
28 }
29
```

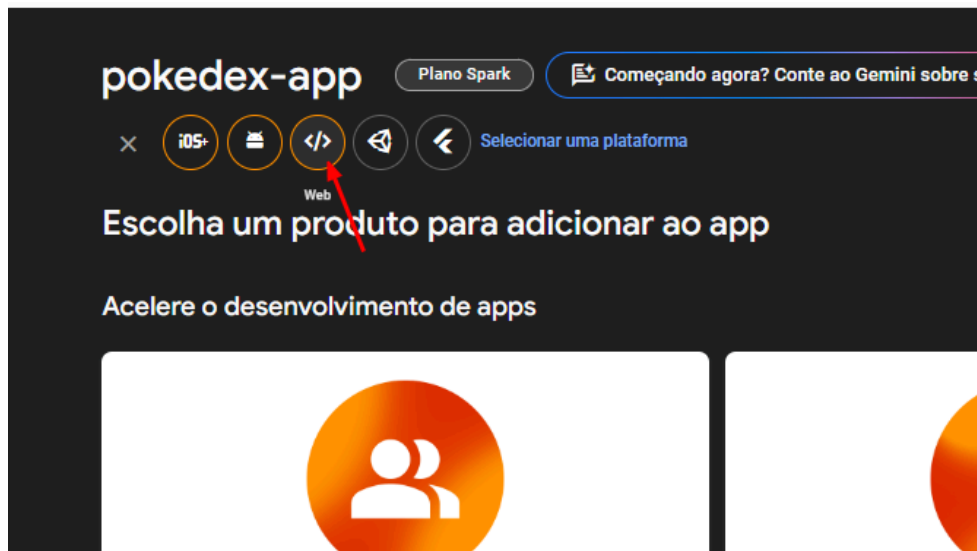

48 - Tente fazer o Hot Reload, veja que o simulador não consegue exibir a tela. Isso acontece pois definimos no main.dart que é necessário inicializar o Firebase, porém, nós não configuramos o Firebase para versão web ao qual o simulador executa.



49 - Volte ao Firebase e clique em + Adicionar app:



50 - Clique em Web



51 - Dê um apelido e clique em Registrar app:

× Adicionar o Firebase ao seu app da Web

1 Registrar app

Apelido do app ?

☐ Configure também o **Firebase Hosting** para este app. [Saiba mais](#)

O Hosting pode ser configurado a qualquer momento. Faça isso a quando quiser sem custos financeiros.

Registrar app

2 Adicionar o SDK do Firebase

52 - Copie as linhas abaixo:

× Adicionar o Firebase ao seu app da Web

✓ Registrar app

2 Adicionar o SDK do Firebase

☒ Usar o npm ☐ Usar a tag <script>

Se você já estiver usando o [npm](#) e um bundler de módulos, como [webpack](#) ou [Rollup](#), execute o seguinte comando para instalar o SDK mais recente ([Saiba mais](#)):

```
$ npm install firebase
```

Depois inicie o Firebase e comece a usar os SDKs dos produtos.

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
const firebaseConfig = {
  apiKey: "AIzaSyCPWZ-xbasRaDExcUvYNb2-I1zPBcKizYg",
  authDomain: "pokedex-app-12f28.firebaseio.com",
  projectId: "pokedex-app-12f28",
  storageBucket: "pokedex-app-12f28.firebaseio.com",
  messagingSenderId: "147417047798",
  appId: "1:147417047798:web:815713eb19374ac7da02d7"
};

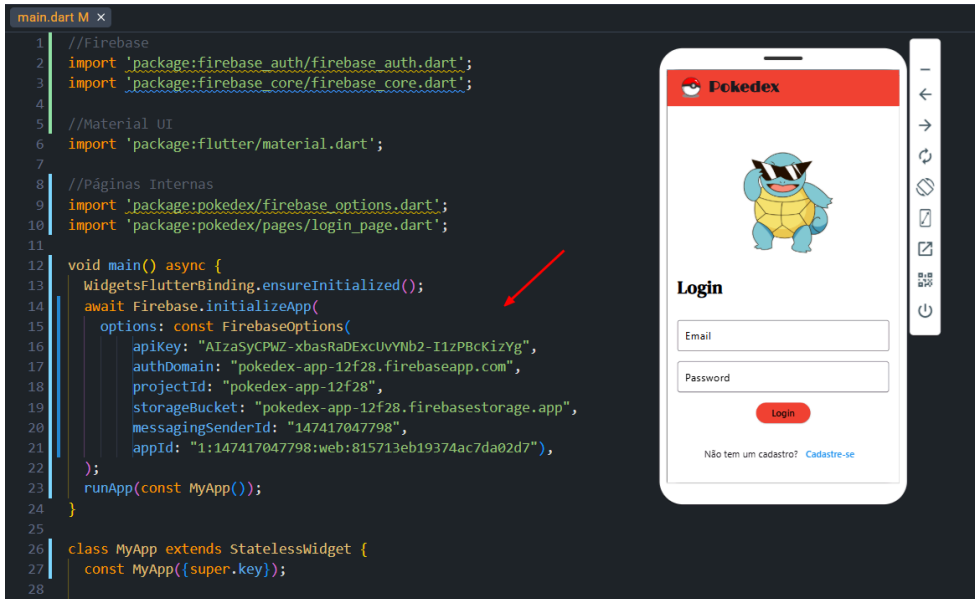
// Initialize Firebase
const app = initializeApp(firebaseConfig);
```

Observação: essa opção usa o [SDK modular para JavaScript](#), que reduz o tamanho do SDK.

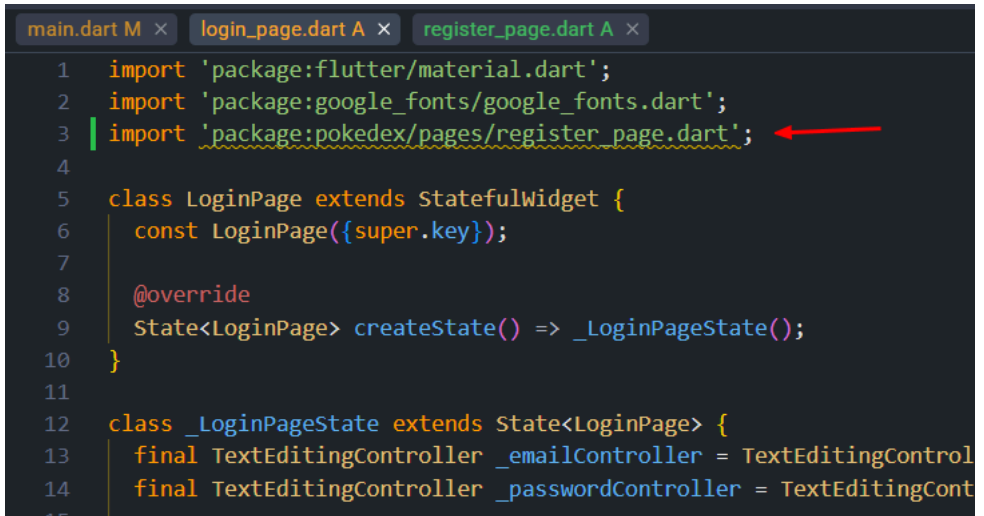
Saiba mais sobre o Firebase para Web: [Vamos começar](#), [Referência da API Web SDK](#), [Amostras](#)

Continuar no console

53 - Faça a seguinte modificação no main para que quando iniciarmos o simulador web podemos ver a execução do app. Ao fazer o Hot Reload a seguinte tela deve aparecer:



54 - Abra o arquivo login_page.dart, vamos fazer o efeito de troca entre as telas. Na linha 3 faça a importação do register_page.dart:

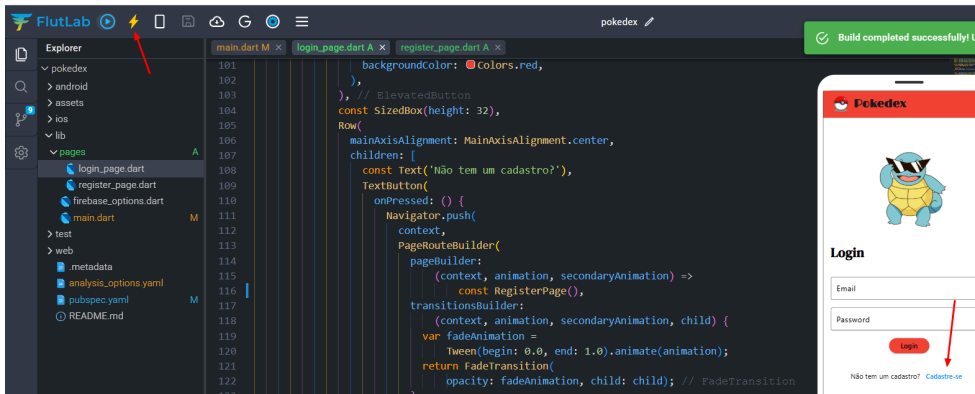


```
main.dart M × login_page.dart A × register_page.dart A ×
1  import 'package:flutter/material.dart';
2  import 'package:google_fonts/google_fonts.dart';
3  import 'package:pokedex/pages/register_page.dart';
4
5  class LoginPage extends StatefulWidget {
6    const LoginPage({super.key});
7
8    @override
9    State<LoginPage> createState() => _LoginPageState();
10 }
11
12 class _LoginPageState extends State<LoginPage> {
13   final TextEditingController _emailController = TextEditingController
14   final TextEditingController _passwordController = TextEditingController
15 }
```

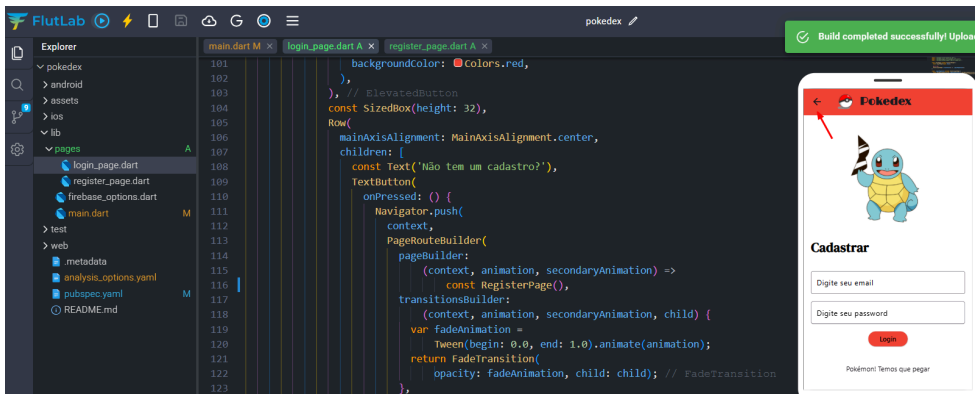
55 - Dentro da função onPressed (linha 110) vamos usar um Navigator.push para linkar as duas páginas, com o PageRouteBuilder podemos construir a rota para a tela de Cadastro:

```
104     const SizedBox(height: 32),
105     Row(
106       mainAxisAlignment: MainAxisAlignment.center,
107       children: [
108         const Text('Não tem um cadastro?'),
109         TextButton(
110           onPressed: () {
111             Navigator.push(
112               context,
113               PageRouteBuilder(
114                 pageBuilder:
115                   (context, animation, secondaryAnimation) =>
116                     const RegisterPage(),
117                 transitionsBuilder:
118                   (context, animation, secondaryAnimation, child) {
119                     var fadeAnimation =
120                       Tween(begin: 0.0, end: 1.0).animate(animation);
121                     return FadeTransition(
122                       opacity: fadeAnimation, child: child); // FadeTransition
123                   },
124                 ), // PageRouteBuilder
125               );
126         },
127         child: const Text(
128           'Cadastre-se',
129           style: TextStyle(color: Colors.blue),
130         ), // Text
131       ), // TextButton
```

56 - Faça o Hot Reload, clique em Cadastre-se no app:

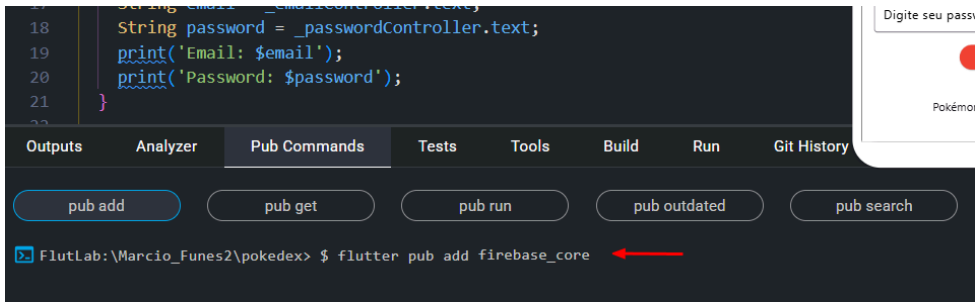


57 - Veja que foi adicionado automaticamente uma seta para voltar a tela de origem da rota:

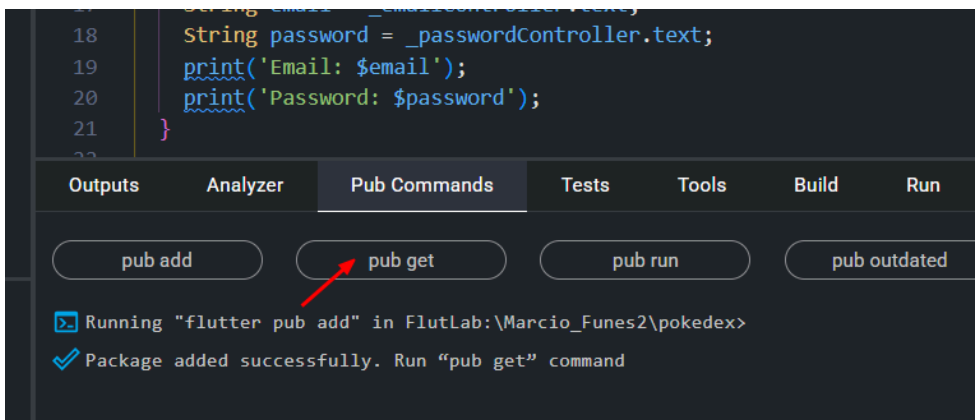


Cadastrando um usuário no Firebase

58 - Vamos instalar alguns pacotes do Firebase para iniciar a conexão, no Pub Commands digite `firebase_core` e dê Enter:



59 - Clique no botão `pub get`:



60 - No arquivo register_page.dart e vamos importar o firebase_auth:

```
register_page.dart x
1  import 'package:firebase_auth/firebase_auth.dart';
2  import 'package:flutter/material.dart';
3  import 'package:google_fonts/google_fonts.dart';
4
5  class RegisterPage extends StatefulWidget {
6    const RegisterPage({super.key});
7
8    @override
9    State<RegisterPage> createState() => _RegisterPageState();
10 }
11
12 class _RegisterPageState extends State<RegisterPage> {
13   final TextEditingController _emailController = TextEditingController();
14   final TextEditingController _passwordController = TextEditingController();
15 }
```

61 - Vamos colocar uma função vazia para evitar erros enquanto não criamos a função de cadastro:

```
98
99
100
101
102
103
104
const SizedBox(height: 16),
ElevatedButton(
  onPressed: () {},
  child: const Text(
    'Cadastrar',
    style: TextStyle(color: Colors.black),
  ), // Text
```

62 - Vamos criar uma função que irá verificar os dados inseridos pelo usuário na tela de cadastro, e irá enviar os dados ao firebase. Na linha 16 crie a função `_register()`:

```
register_page.dart A x
1  import 'package:firebase_auth/firebase_auth.dart';
2  import 'package:flutter/material.dart';
3  import 'package:google_fonts/google_fonts.dart';
4
5  class RegisterPage extends StatefulWidget {
6    const RegisterPage({super.key});
7
8    @override
9    State<RegisterPage> createState() => _RegisterPageState();
10 }
11
12 class _RegisterPageState extends State<RegisterPage> {
13   final TextEditingController _emailController = TextEditingController();
14   final TextEditingController _passwordController = TextEditingController();
15   final FirebaseAuth _auth = FirebaseAuth.instance;
16
17   _register() async {
18     String email = _emailController.text.trim();
19     String password = _passwordController.text.trim();
20
21     if (email.isEmpty || password.isEmpty) {
22       ScaffoldMessenger.of(context).showSnackBar(
23         const SnackBar(content: Text('Preencha todos os campos!')),
24       );
25       return;
26     }
27 }
```

Feedback com SnackBar

63 - Aqui criamos uma verificação se o e-mail está corretamente formatado e também um SnackBar, mensagem de feedback ao usuário:

<https://docs.flutter.dev/cookbook/design/snackbars>

```
25     return;
26   }
27
28   if (!RegExp(r'^^[^@]+@[^\s]+\.[^\s]+').hasMatch(email)) {
29     ScaffoldMessenger.of(context).showSnackBar(
30       const SnackBar(content: Text('Digite um email válido!')),
31     );
32     return;
33   }
34
35   try {
36     UserCredential userCredential =
37       await _auth.createUserWithEmailAndPassword(
38         email: email,
39         password: password,
40       );
41     ScaffoldMessenger.of(context).showSnackBar(
42       SnackBar(
43         content: Text('Usuário cadastrado: ${userCredential.user!.email}')), // SnackBar
44     );
45   } catch (e) {
46     ScaffoldMessenger.of(context).showSnackBar(
47       SnackBar(content: Text('Erro ao cadastrar: $e')),
48     );
49   }
50 }
51
52 @override
```

64 - Agora que criamos a função `_register`, chame-a quando o usuário pressionar o botão Cadastrar:

```
121     ), // TextField
122     const SizedBox(height: 16),
123     ElevatedButton(
124       onPressed: _register,
125       child: const Text(
126         'Cadastrar',
127         style: TextStyle(color: Colors.black),
128       ), // Text
129       style: ElevatedButton.styleFrom(
130         backgroundColor: Colors.red,
131       ),
132     ),
```

65 - Veja que ao tentar colocar qualquer informação ou nenhuma no e-mail o `SnackBar` informa ao usuário:



66 - Faça um cadastro real:

Cadastrar

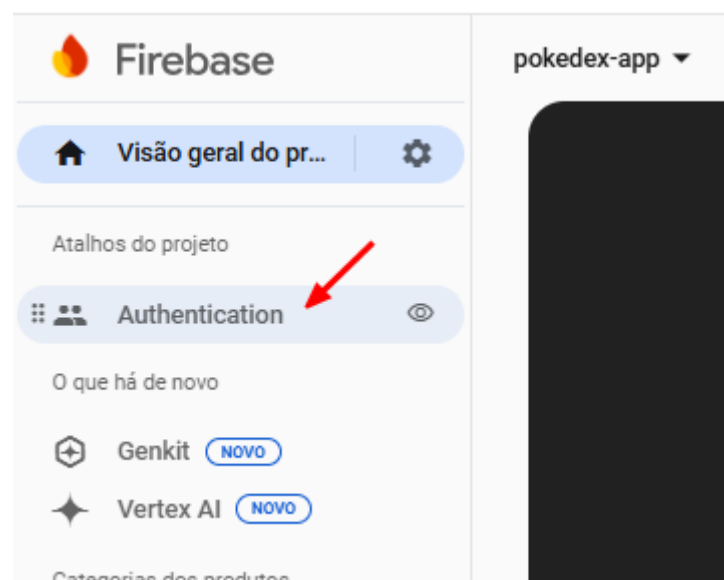
Digite seu email

Digite seu password

Cadastrar

Usuário cadastrado: marciofunes@facef.br

67 - No Firebase clique em Authentication



68 - Veja que o usuário foi cadastrado com sucesso!

Visão geral do projeto

Azuleiros do projeto

Authentication

O que há de novo

Genkit

Vertex AI

Categorias dos produtos

Criação

Executar

Analytics

AI

pokedex-app

Authentication

Usuários

Método de login

Modelos

Uso

Configurações

Extensões

Os seguintes recursos de autenticação vão parar de funcionar quando o Firebase Dynamic Links for desativado em 25 de agosto de 2025: autenticação por link de e-mail em apps para dispositivos móveis e suporte ao OAuth do Cordova em apps da Web.

Pesquise por endereço de e-mail, número de telefone ou UID do usuário

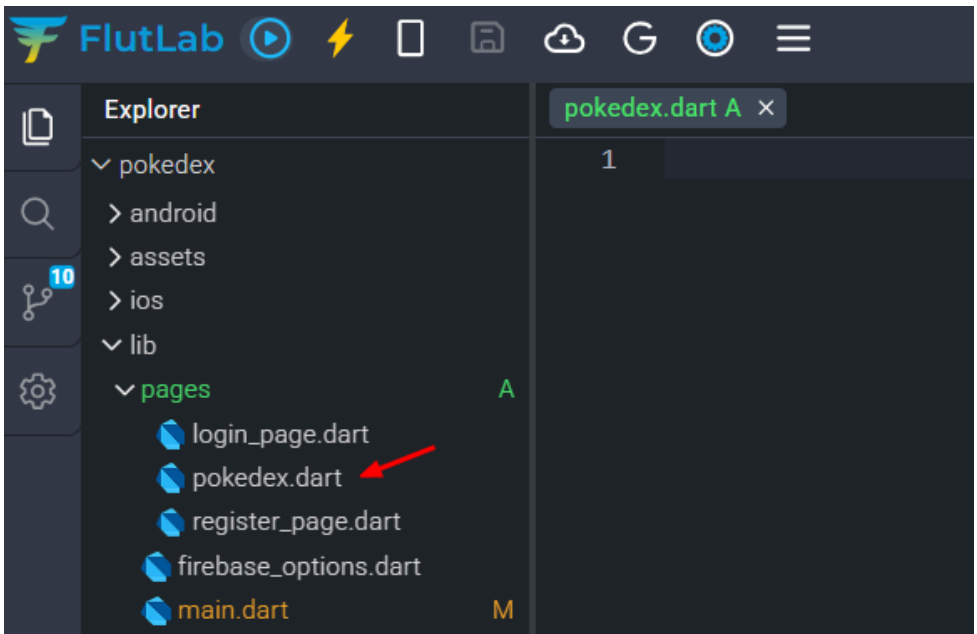
Adicionar usuário

Identificador	Provedores	Data de criação	Último login	UID do usuário
marciofunes@facef.br		29 de mar. de 20...	29 de mar. de 20...	tyRQ25mPkgH-IELEuW7K0Z...

Linhas por página: 50 1 - 1 of 1

Criando a tela da Pokédex

69 - Antes de configurarmos o login do usuário usando o cadastro feito no passo anterior, precisamos criar a tela ao qual o aplicativo deverá enviar o usuário após o login. Na pasta pages crie um arquivo chamado pokedex.dart:



70 - Está será a estrutura da nossa Pokedex:

```
pokedex.dart M x
1  import 'package:google_fonts/google_fonts.dart';
2  import 'package:flutter/material.dart';
3  import 'package:http/http.dart' as http;
4  import 'dart:convert';
5
6  > void main() { ...
8  }
9
10 > class PokedexApp extends StatelessWidget { ...
19   }
20
21 > class PokedexScreen extends StatefulWidget { ...
26   }
27
28 > class _PokedexScreenState extends State<PokedexScreen> { ...
173  }
174
175 > class PokemonDetailScreen extends StatelessWidget { ...
279   }
280
281 |
```

80 - Faça o main que irá instanciar nossa classe PokedexApp e a classe PokedexApp que exibe como home os componentes do PokedexScreen:

```
1  import 'package:google_fonts/google_fonts.dart';
2  import 'package:flutter/material.dart';
3  import 'package:http/http.dart' as http;
4  import 'dart:convert';
5
6  void main() {
7    runApp(const PokedexApp());
8  }
9
10 class PokedexApp extends StatelessWidget {
11   const PokedexApp({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     return const MaterialApp(
16       home: PokedexScreen(),
17     ); // MaterialApp
18   }
19 }
20
21 > class PokedexScreen extends StatefulWidget { ...
26   }
27
28 > class _PokedexScreenState extends State<PokedexScreen> { ...
173   }
174
175 > class PokemonDetailScreen extends StatelessWidget { ...
279   }
280
281 |
```

81 - Adicione um StatefulWidget chamado PokedexScreen, isso significa que essa tela pode ter um estado interno que muda durante a execução do aplicativo:

```
1  import 'package:google_fonts/google_fonts.dart';
2  import 'package:flutter/material.dart';
3  import 'package:http/http.dart' as http;
4  import 'dart:convert';
5
6  > void main() { ...
8  }
9
10 > class PokedexApp extends StatelessWidget { ...
19 }
20
21 class PokedexScreen extends StatefulWidget {
22   const PokedexScreen({super.key});
23
24   @override
25   PokedexScreenState createState() => _PokedexScreenState();
26 }
27
28 > class _PokedexScreenState extends State<PokedexScreen> { ...
173 }
174
175 > class PokemonDetailScreen extends StatelessWidget { ...
279 }
280
281 | |
```



82 - Aqui vamos criar um método chamado fetchPokemon que faz uma requisição à PokéAPI para buscar uma lista de Pokémon:

```
28 class _PokedexScreenState extends State<PokedexScreen> {
29     List<dynamic> pokemonList = [];
30     List<dynamic> filteredPokemonList = [];
31     TextEditingController searchController = TextEditingController();
32
33     @override
34     void initState() {
35         super.initState();
36         fetchPokemon();
37     }
38
39     Future<void> fetchPokemon() async {
40         try {
41             final response = await http.get(
42                 Uri.parse('https://pokeapi.co/api/v2/pokemon?limit=150'),
43             );
44
45             if (response.statusCode == 200) {
46                 final data = jsonDecode(response.body);
47                 setState(() {
48                     pokemonList = data['results'];
49                     filteredPokemonList = pokemonList;
50                 });
51             } else {
52                 throw Exception('Erro ao carregar Pokémon');
53             }
54         } catch (e) {
55             print('Erro na requisição: $e');
56         }
57     }
58 }
```

83 - Outro método chamado `filterPokemon` que filtra a lista de Pokémon com base no nome digitado pelo usuário, ignorando maiúsculas e minúsculas:

```
58
59 void filterPokemon(String query) {
60     setState(() {
61         filteredPokemonList = pokemonList
62         .where((pokemon) =>
63             pokemon['name'].toLowerCase().contains(query.toLowerCase()))
64         .toList();
65     });
66 }
67
```

84 - Começando agora a criar o Scaffold e o App com os componentes iniciais da tela:

```
68   @override
69   Widget build(BuildContext context) {
70     return Scaffold(
71       appBar: AppBar(
72         title: Row(children: [
73           Image.asset(
74             'assets/pokeball.gif',
75             height: 40,
76             width: 40,
77           ), // Image.asset
78           const SizedBox(width: 8),
79           Text(
80             'Pokedex',
81             style: GoogleFonts.limelight(
82               fontSize: 24,
83               fontWeight: FontWeight.bold,
84             ),
85           ), // Text
86         ]), // Row
```

85 - Vamos definir a barra de busca com o searchController:

```
87     elevation: 0,  
88     bottom: PreferredSize(  
89       preferredSize: const Size.fromHeight(50.0),  
90       child: Padding(  
91         padding: const EdgeInsets.all(8.0),  
92         child: TextField(  
93           controller: searchController,  
94           decoration: const InputDecoration(  
95             hintText: 'Buscar Pokémon...',  
96             border: OutlineInputBorder(),  
97             prefixIcon: Icon(Icons.search),  
98           ), // InputDecoration  
99           onChanged: filterPokemon,  
100         ), // TextField  
101       ), // Padding  
102     ), // PreferredSize  
103   ), // AppBar
```

86 - Dentro do body de nossa tela vamos criar um grid para organizar a exibição dos pokémon retornados via API:

```
104     body: filteredPokemonList.isEmpty
105         ? const Center(child: CircularProgressIndicator())
106         : GridView.builder(
107             padding: const EdgeInsets.all(8.0),
108             gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
109                 crossAxisCount: 2,
110                 childAspectRatio: 1.5,
111                 crossAxisSpacing: 8,
112                 mainAxisSpacing: 8,
113             ), // SliverGridDelegateWithFixedCrossAxisCount
114             itemCount: filteredPokemonList.length,
115             itemBuilder: (context, index) {
116                 return FutureBuilder(
117                     future:
118                         http.get(Uri.parse(filteredPokemonList[index]['url'])),
119                     builder: (context, snapshot) {
120                         if (!snapshot.hasData) {
121                             return const Center(child: CircularProgressIndicator());
122                         }
123                         final data = jsonDecode(snapshot.data!.body);
124                         final String gifUrl = data['sprites']['versions']
125                             ['generation-v']['black-white']['animated']
126                             ['front_default'] ??
127                             data['sprites']['front_default'];
128                         final String type = data['types'][0]['type']['name'];
```


87 - Para que a cor do fundo do card seja compatível com o tipo do pokémon, criamos aqui uma lista de referências com o Map associando cada tipo a uma cor, já que essa cor não vem por padrão na API:

```
129 final Map<String, Color> typeColors = {
130     'fire':  Colors.red,
131     'water':  Colors.blue,
132     'grass':  Colors.green,
133     'electric':  Colors.yellow,
134     'bug':  Colors.lightGreen,
135     'normal':  Colors.grey,
136     'poison':  Colors.purple,
137     'ground':  Colors.brown,
138     'fairy':  Colors.pink,
139     'fighting':  Colors.orange,
140     'psychic':  Colors.pinkAccent,
141     'rock':  Colors.brown,
142     'ghost':  Colors.deepPurple,
143     'ice':  Colors.lightBlue,
144     'dragon':  Colors.indigo,
145     'dark':  Colors.black,
146     'steel':  Colors.blueGrey,
147     'flying':  Colors.indigoAccent,
148 }
```

88 - Para finalizar a classe `_PokedexScreenState` vamos utilizar o `GestureDetector` para verificar em qual car o usuário tocou, por último vamos de fato definir a estrutura do card que a medida que novos pokemon vão sendo recebidos via API novos cards irão surgir:

```
149         return GestureDetector(
150             onTap: () {
151                 Navigator.push(
152                     context,
153                     MaterialPageRoute(
154                         builder: (context) => PokemonDetailScreen(
155                             url: filteredPokemonList[index]['url'],
156                             typeColor: typeColors[type] ?? Colors.grey),
157                         // MaterialPageRoute
158                     );
159             },
160             child: Card(
161                 color: typeColors[type] ?? Colors.grey,
162                 child: Column(
163                     mainAxisAlignment: MainAxisAlignment.center,
164                     children: [
165                         Image.network(gifUrl, height: 80),
166                         Text(
167                             filteredPokemonList[index]['name'].toUpperCase(),
168                             style: const TextStyle(
169                                 fontWeight: FontWeight.bold,
170                                 color: Colors.white), // TextStyle
171                         ), // Text
172                     ],
173                 ), // Column
174             ), // Card
175         ); // GestureDetector
176     ],
177 ); // FutureBuilder
178 },
179 ), // GridView.builder
180 ); // Scaffold
181 }
182 }
```

89 - Por fim vamos criar a classe que define o que o usuário irá ver quando clicar em um card:

```
1  import 'package:google_fonts/google_fonts.dart';
2  import 'package:flutter/material.dart';
3  import 'package:http/http.dart' as http;
4  import 'dart:convert';
5
6  > void main() {...
8  }
9
10 > class PokedexApp extends StatelessWidget {...
19 }
20
21 > class PokedexScreen extends StatefulWidget {...
26 }
27
28 > class _PokedexScreenState extends State<PokedexScreen> {...
182 }
183
184 class PokemonDetailScreen extends StatelessWidget {
185   final String url;
186   final Color typeColor;
187
188   const PokemonDetailScreen(
189     {super.key, required this.url, required this.typeColor});
190
191   Future<Map<String, dynamic>> fetchPokemonDetails() async {
192     final response = await http.get(Uri.parse(url));
193     if (response.statusCode == 200) {
194       return jsonDecode(response.body);
195     }
196     throw Exception('Erro ao carregar dados');
197   }
198
```



90 - Como todo inicia de tela vamos definir um Scaffold e um AppBar, teremos um botão de voltar (arrow_back):

```
199 @override
200 Widget build(BuildContext context) {
201   return Scaffold(
202     appBar: AppBar(
203       backgroundColor: typeColor,
204       leading: IconButton(
205         icon: const Icon(Icons.arrow_back),
206         onPressed: () {
207           Navigator.pop(context);
208         },
209       ), // IconButton
210     ), // AppBar
211     body: FutureBuilder(
212       future: fetchPokemonDetails(),
213       builder: (context, snapshot) {
214         if (!snapshot.hasData) {
215           return const Center(child: CircularProgressIndicator());
216         }
217         final pokemonData = snapshot.data!;
218         final String gifUrl = pokemonData['sprites']['versions']
219           ['generation-v']['black-white']['animated']
220           ['front_default'] ??
221           pokemonData['sprites']['front_default'];
```

91 - Vamos criar um Container que irá definir o tamanho do pokemon na tela de detalhes, seu nome, peso e altura:

```
222 return Container(  
223   color: typeColor,  
224   child: Center(  
225     child: SingleChildScrollView(  
226       child: Column(  
227         mainAxisAlignment: MainAxisAlignment.center,  
228         children: [  
229           SizedBox(  
230             height: 150,  
231             width: 150,  
232             child: Image.network(  
233               gifUrl,  
234               fit: BoxFit.contain,  
235             ), // Image.network  
236           ), // SizedBox  
237           Text(  
238             pokemonData['name'].toUpperCase(),  
239             style: const TextStyle(  
240               fontSize: 28,  
241               fontWeight: FontWeight.bold,  
242               color: Colors.white), // TextStyle  
243           ), // Text  
244           Text(  
245             'Altura: ${(pokemonData['height'] / 10).toStringAsFixed(1)} m',  
246             style:  
247               const TextStyle(fontSize: 18, color: Colors.white)), // Text  
248           Text(  
249             'Peso: ${(pokemonData['weight'] / 10).toStringAsFixed(1)} kg',  
250             style:  
251               const TextStyle(fontSize: 18, color: Colors.white)), // Text
```

92 - Finalizando com as informações do pokémon como tipos, habilidades e etc:

```
251         const TextStyle(fontSize: 18, color: Colors.white)), // Text
252     const SizedBox(height: 10),
253     const Text('Tipos:',
254       style: TextStyle(
255         fontSize: 20,
256         fontWeight: FontWeight.bold,
257         color: Colors.white)), // TextStyle // Text
258     Wrap(
259       spacing: 10,
260       children: (pokemonData['types'] as List)
261         .map((type) => Chip(
262           label: Text(type['type']['name'].toUpperCase())) // Chip
263         .toList(),
264     ), // Wrap
265     const SizedBox(height: 10),
266     const Text('Habilidades:',
267       style: TextStyle(
268         fontSize: 20,
269         fontWeight: FontWeight.bold,
270         color: Colors.white)), // TextStyle // Text
271     Wrap(
272       spacing: 10,
273       children: (pokemonData['abilities'] as List)
274         .map((ability) => Chip(
275           label: Text(
276             ability['ability']['name'].toUpperCase())) // Text // Chip
277         .toList(),
278     ), // Wrap
279   ],
280 ), // Column
281 ), // SingleChildScrollView
282 ), // Center
283 ); // Container
284 },
285 ), // FutureBuilder
286 ); // Scaffold
287 }
288 }
289 }
```

Realizando login via Firebase

93 - Agora vamos validar o usuário já cadastrado e permitir ou não o login, faça a importação do `firebase_auth` e também da tela que o usuário deve ser levado após o login (`pokedex.dart`):

```
1  import 'package:flutter/material.dart';
2  import 'package:google_fonts/google_fonts.dart';
3  import 'package:pokedex/pages/register_page.dart';
4  import 'package:firebase_auth/firebase_auth.dart';
5  import 'package:pokedex/pages/pokedex.dart';
6
7  class LoginPage extends StatefulWidget {
8    const LoginPage({super.key});
9
10   @override
11   State<LoginPage> createState() => _LoginPageState();
12 }
13
14 class _LoginPageState extends State<LoginPage> {
15   final TextEditingController _emailController = TextEditingController();
16   final TextEditingController _passwordController = TextEditingController();
17   bool _isLoading = false;
18 }
```

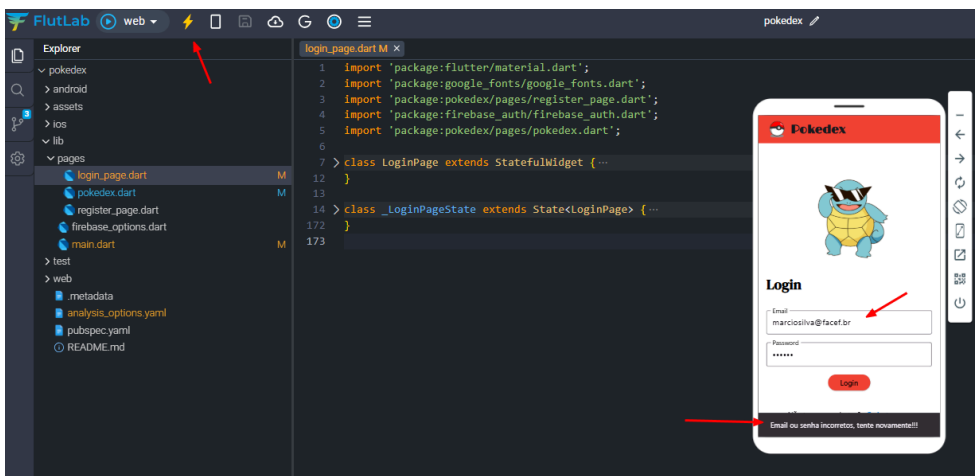
94 - Vamos agora modificar nossa função `_login()` que antes apenas printava email e senha no console para de fato realizar o login via autenticação do firebase:

```
14 class _LoginPageState extends State<LoginPage> {
15   final TextEditingController _emailController = TextEditingController();
16   final TextEditingController _passwordController = TextEditingController();
17   bool _isLoading = false;
18
19   void _login() async {
20     String email = _emailController.text.trim();
21     String password = _passwordController.text.trim();
22
23     if (email.isEmpty || password.isEmpty) {
24       ScaffoldMessenger.of(context).showSnackBar(
25         const SnackBar(content: Text('Por favor, preencha todos os campos')),
26       );
27       return;
28     }
29
30     setState(() => _isLoading = true);
31
32     try {
33       await FirebaseAuth.instance.signInWithEmailAndPassword(
34         email: email,
35         password: password,
36       );
37       // Redireciona para a PokedexApp
38       Navigator.pushReplacement(
39         context,
40         MaterialPageRoute(builder: (context) => const PokedexApp()),
41       );
42     } on FirebaseAuthException catch (e) {
43       ScaffoldMessenger.of(context).showSnackBar(
44         const SnackBar(
45           content: Text('Email ou senha incorretos, tente novamente!!!'),
46         ),
47     ) finally {
48       setState(() => _isLoading = false);
49     }
50   }
51 }
```

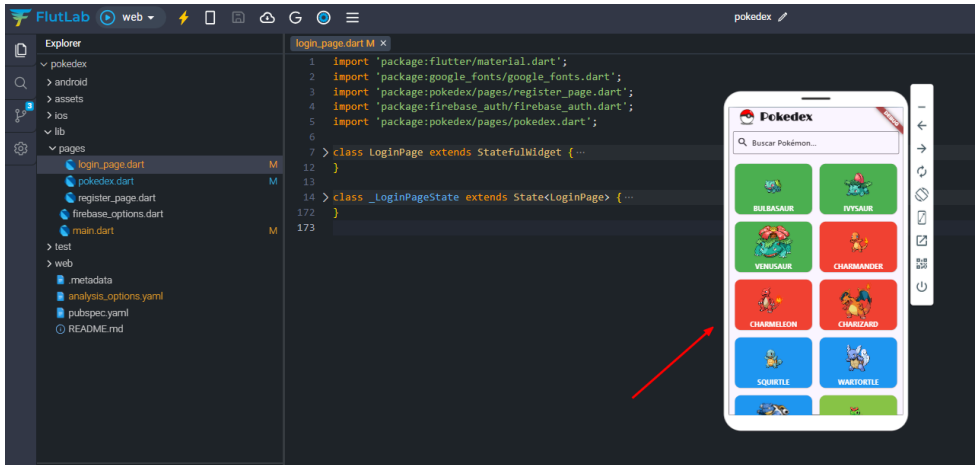

95 - Agora precisamos chamar essa função ao botão de login ser pressionado, vamos adicionar um indicador de progresso circular para demonstrar ao usuário que a autenticação está em andamento:

```
123 ElevatedButton(  
124   onPressed: _isLoading ? null : _login,  
125   child: isLoading  
126     ? const CircularProgressIndicator(color: Colors.white)  
127     : const Text(  
128       'Login',  
129       style: TextStyle(color: Colors.black),  
130     ), // Text  
131   style: ElevatedButton.styleFrom(  
132     backgroundColor: Colors.red,  
133     disabledBackgroundColor: Colors.grey,  
134   ),  
135 ), // ElevatedButton
```

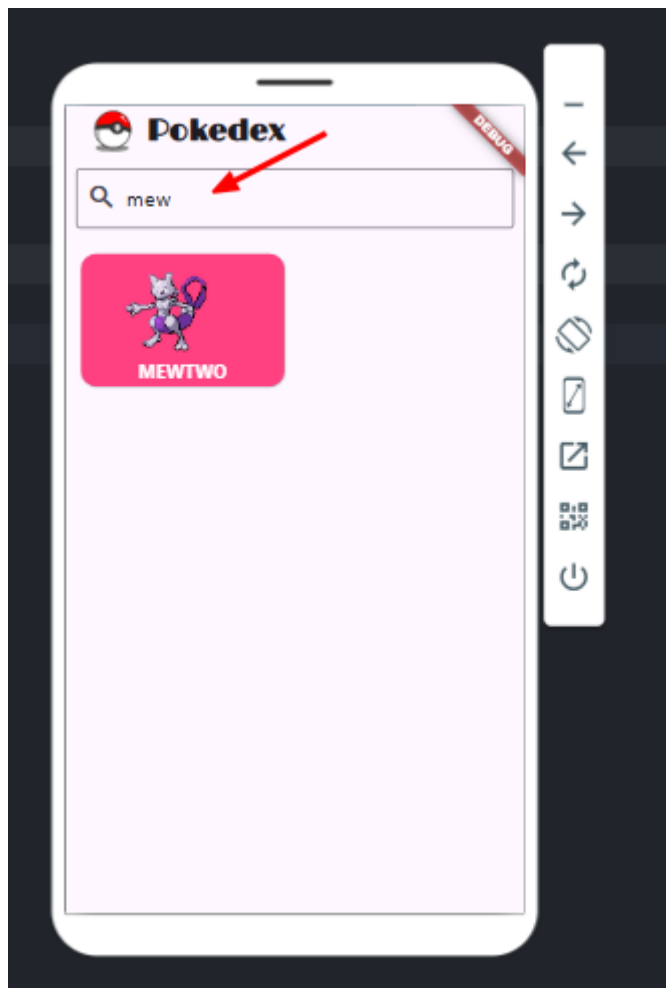
96 - Faça o Hot Reload e coloque um e-mail errado, veja que a validação aconteceu:



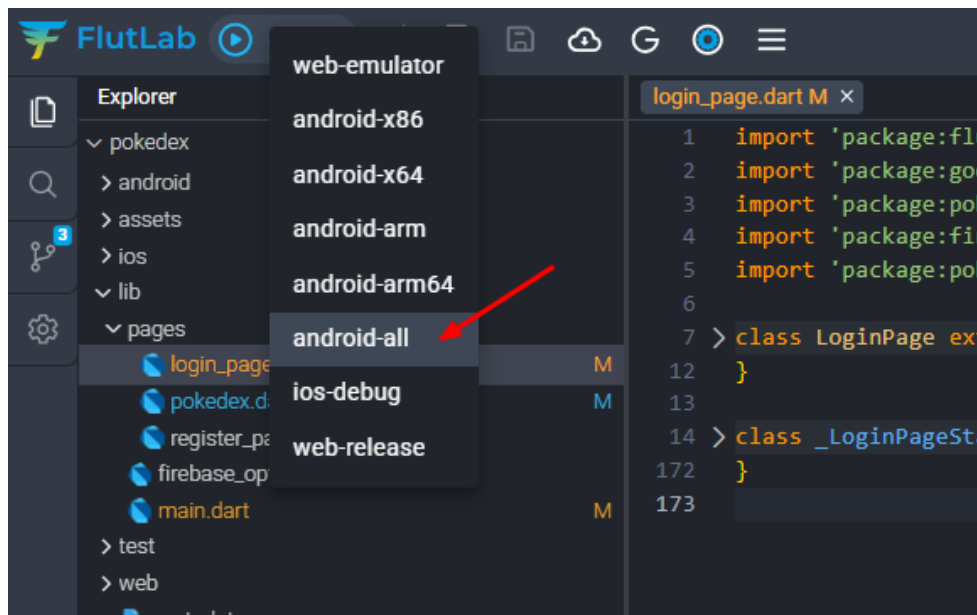
97 - Faça o login corretamente, o usuário deve ser levado a visualizar a Pokédex corretamente:



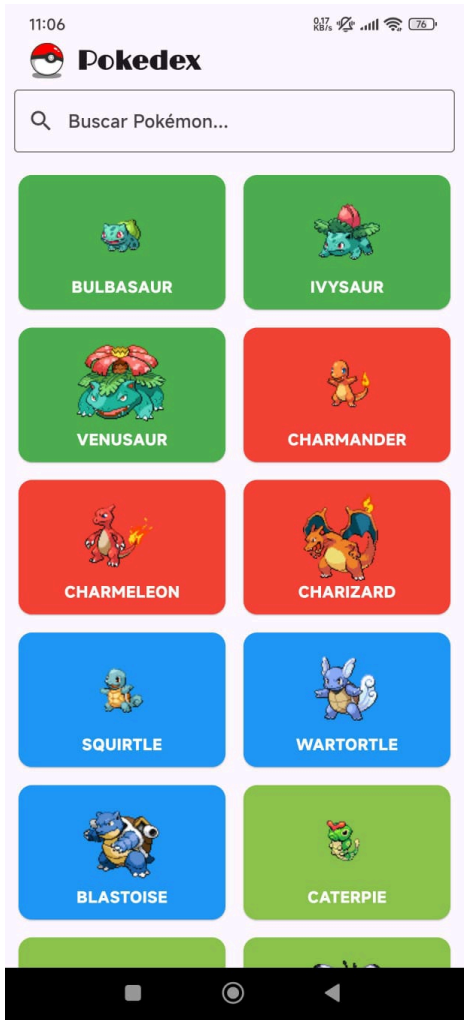
98 - Faça um teste da barra de busca se verifique se está funcional:



99 - Faça a build para a plataforma Android ou IOS:



100 - Parabéns, agora é só testar em seu smartphone, faça cadastro e login:



Código-fonte

main.dart

```
//Firebase
import 'package:firebase_auth/firebase_auth.dart';
import 'package:firebase_core/firebase_core.dart';

//Material UI
import 'package:flutter/material.dart';

//Páginas Internas
import 'package:pokedex/firebase_options.dart';
import 'package:pokedex/pages/login_page.dart';

import 'package:pokedex/pages/pokedex.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: const FirebaseOptions(
      apiKey: "AlzaSyCPWZ-xbasRaDExcUvYNb2-l1zPBcKizYg",
      authDomain: "pokedex-app-12f28.firebaseio.com",
      projectId: "pokedex-app-12f28",
      storageBucket: "pokedex-app-12f28.firebaseio.com",
      messagingSenderId: "147417047798",
      appId: "1:147417047798:web:815713eb19374ac7da02d7"),
  );
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

```
@override
Widget build(BuildContext context) {
  return const MaterialApp(
    debugShowCheckedModeBanner: false,
    home: LoginPage(),
  );
}
```

login_page.dart

```
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';
import 'package:pokedex/pages/register_page.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:pokedex/pages/pokedex.dart';

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});

  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final TextEditingController _emailController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();
  bool _isLoading = false;

  void _login() async {
    String email = _emailController.text.trim();
    String password = _passwordController.text.trim();

    if (email.isEmpty || password.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Por favor, preencha todos os campos')),
      );
      return;
    }

    setState(() => _isLoading = true);

    try {
```



```

await FirebaseAuth.instance.signInWithEmailAndPassword(
  email: email,
  password: password,
);
// Redireciona para a PokedexApp
Navigator.pushReplacement(
  context,
  MaterialPageRoute(builder: (context) => const PokedexApp()),
);
} on FirebaseAuthException catch (e) {
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('Email ou senha incorretos, tente novamente!!!'),
    );
  } finally {
    setState(() => _isLoading = false);
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Row(
        children: [
          Image.asset(
            'assets/pokeball.gif',
            height: 40,
            width: 40,
          ),
          const SizedBox(width: 8),
          Text(
            'Pokedex',
            style: GoogleFonts.limelight(
              fontSize: 24,

```

```

        fontWeight: FontWeight.bold,
      ),
    ),
  ],
),
backgroundColor: Colors.red,
elevation: 0,
),
body: Container(
  color: Colors.white,
  child: Padding(
    padding: const EdgeInsets.all(16.0),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: <Widget>[
        Center(
          child: Image.asset(
            'assets/squirtle1.png',
            height: 200,
            width: 200,
          ),
        ),
      ],
      const SizedBox(height: 32),
      Align(
        alignment: Alignment.centerLeft,
        child: Text(
          'Login',
          style: GoogleFonts.dmSerifText(
            fontSize: 28,
            fontWeight: FontWeight.bold,
            color: Colors.black,
          ),
        ),
      ),
      const SizedBox(height: 32),

```

```

TextField(
  controller: _emailController,
  decoration: const InputDecoration(
    labelText: 'Email',
    labelStyle: TextStyle(color: Colors.black),
    border: OutlineInputBorder(),
  ),
  keyboardType: TextInputType.emailAddress,
),
const SizedBox(height: 16),
TextField(
  controller: _passwordController,
  decoration: const InputDecoration(
    labelText: 'Password',
    labelStyle: TextStyle(color: Colors.black),
    border: OutlineInputBorder(),
  ),
  obscureText: true,
),
const SizedBox(height: 16),
ElevatedButton(
  onPressed: _isLoading ? null : _login,
  child: _isLoading
    ? const CircularProgressIndicator(color: Colors.white)
    : const Text(
        'Login',
        style: TextStyle(color: Colors.black),
      ),
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.red,
    disabledBackgroundColor: Colors.grey,
  ),
),
const SizedBox(height: 32),
Row(

```

```

mainAxisAlignment: MainAxisAlignment.center,
children: [
  const Text('Não tem um cadastro?'),
  TextButton(
    onPressed: () {
      Navigator.push(
        context,
        PageRouteBuilder(
          pageBuilder:
            (context, animation, secondaryAnimation) =>
              const RegisterPage(),
          transitionsBuilder:
            (context, animation, secondaryAnimation, child) {
              var fadeAnimation =
                Tween(begin: 0.0, end: 1.0).animate(animation);
              return FadeTransition(
                opacity: fadeAnimation, child: child);
            },
          ),
        );
    },
    child: const Text(
      'Cadastre-se',
      style: TextStyle(color: Colors.blue),
    ),
  ),
],
),
],
),
),
),
);
}
}

```

register_page.dart

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

class RegisterPage extends StatefulWidget {
  const RegisterPage({super.key});

  @override
  State<RegisterPage> createState() => _RegisterPageState();
}

class _RegisterPageState extends State<RegisterPage> {
  final TextEditingController _emailController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();
  final FirebaseAuth _auth = FirebaseAuth.instance;

  _register() async {
    String email = _emailController.text.trim();
    String password = _passwordController.text.trim();

    if (email.isEmpty || password.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Preencha todos os campos!')),
      );
      return;
    }

    if (!RegExp(r'^^[^@]+@^[^@]+\.[^@]+').hasMatch(email)) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Digite um email válido!')),
      );
      return;
    }
  }
}
```

```

try {
  UserCredential userCredential =
    await _auth.createUserWithEmailAndPassword(
      email: email,
      password: password,
    );
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text('Usuário cadastrado: ${userCredential.user!.email}')),
    );
} catch (e) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Erro ao cadastrar: $e')),
  );
}
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Row(
        children: [
          Image.asset(
            'assets/pokeball.gif',
            height: 40,
            width: 40,
          ),
          const SizedBox(width: 8),
          Text(
            'Pokedex',
            style: GoogleFonts.limelight(
              fontSize: 24,
              fontWeight: FontWeight.bold,
            ),
          ),
        ],
      ),
    ),
  );
}

```

```

    ),
  ),
],
),
backgroundColor: Colors.red,
elevation: 0,
),
body: Container(
  color: Colors.white,
  child: Padding(
    padding: const EdgeInsets.all(16.0),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: <Widget>[
        Center(
          child: Image.asset(
            'assets/squirtle2.png',
            height: 200,
            width: 200,
          ),
        ),
        const SizedBox(height: 32),
        Align(
          alignment: Alignment.centerLeft,
          child: Text(
            'Cadastrar',
            style: GoogleFonts.dmSerifText(
              fontSize: 28,
              fontWeight: FontWeight.bold,
              color: Colors.black,
            ),
          ),
        ),
        const SizedBox(height: 32),
        TextField(

```

```
controller: _emailController,
decoration: const InputDecoration(
  labelText: 'Digite seu email',
  labelStyle: TextStyle(color: Colors.black),
  border: OutlineInputBorder(),
),
keyboardType: TextInputType.emailAddress,
),
const SizedBox(height: 16),
TextField(
  controller: _passwordController,
  decoration: const InputDecoration(
    labelText: 'Digite seu password',
    labelStyle: TextStyle(color: Colors.black),
    border: OutlineInputBorder(),
  ),
  obscureText: true,
),
const SizedBox(height: 16),
ElevatedButton(
  onPressed: _register,
  child: const Text(
    'Cadastrar',
    style: TextStyle(color: Colors.black),
  ),
  style: ElevatedButton.styleFrom(
    backgroundColor: Colors.red,
  ),
),
const SizedBox(height: 32),
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    const Text('Pokémon! Temos que pegar'),
    const SizedBox(height: 32),
```



```
        1,  
    )  
    1,  
    ),  
    ),  
    ),  
    );  
}
```

```
}
```

pokedex.dart

```
import 'package:google_fonts/google_fonts.dart';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';
```

```
void main() {
  runApp(const PokedexApp());
}
```

```
class PokedexApp extends StatelessWidget {
  const PokedexApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: PokedexScreen(),
    );
  }
}
```

```
class PokedexScreen extends StatefulWidget {
  const PokedexScreen({super.key});
```

```
  @override
  _PokedexScreenState createState() => _PokedexScreenState();
}
```

```
class _PokedexScreenState extends State<PokedexScreen> {
  List<dynamic> pokemonList = [];
  List<dynamic> filteredPokemonList = [];
  TextEditingController searchController = TextEditingController();
```

```
  @override
```

```

void initState() {
  super.initState();
  fetchPokemon();
}

```

```

Future<void> fetchPokemon() async {
  try {
    final response = await http.get(
      Uri.parse('https://pokeapi.co/api/v2/pokemon?limit=150'),
    );

    if (response.statusCode == 200) {
      final data = jsonDecode(response.body);
      setState(() {
        pokemonList = data['results'];
        filteredPokemonList = pokemonList;
      });
    } else {
      throw Exception('Erro ao carregar Pokémon');
    }
  } catch (e) {
    print('Erro na requisição: $e');
  }
}

```

```

void filterPokemon(String query) {
  setState(() {
    filteredPokemonList = pokemonList
      .where((pokemon) =>
        pokemon['name'].toLowerCase().contains(query.toLowerCase()))
      .toList();
  });
}

```

@override

```
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Row(children: [
        Image.asset(
          'assets/pokeball.gif',
          height: 40,
          width: 40,
        ),
        const SizedBox(width: 8),
        Text(
          'Pokedex',
          style: GoogleFonts.limelight(
            fontSize: 24,
            fontWeight: FontWeight.bold,
          ),
        ),
      ]),
      elevation: 0,
      bottom: PreferredSize(
        preferredSize: const Size.fromHeight(50.0),
        child: Padding(
          padding: const EdgeInsets.all(8.0),
          child: TextField(
            controller: searchController,
            decoration: const InputDecoration(
              hintText: 'Buscar Pokémon...',
              border: OutlineInputBorder(),
              prefixIcon: Icon(Icons.search),
            ),
            onChanged: filterPokemon,
          ),
        ),
      ),
    ),
  ),
),
),
```

```

body: filteredPokemonList.isEmpty
    ? const Center(child: CircularProgressIndicator())
    : GridView.builder(
        padding: const EdgeInsets.all(8.0),
        gridDelegate: const
SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
    childAspectRatio: 1.5,
    crossAxisSpacing: 8,
    mainAxisSpacing: 8,
),
    itemCount: filteredPokemonList.length,
    itemBuilder: (context, index) {
        return FutureBuilder(
            future:
                http.get(Uri.parse(filteredPokemonList[index]['url'])),
            builder: (context, snapshot) {
                if (!snapshot.hasData) {
                    return const Center(child: CircularProgressIndicator());
                }
                final data = jsonDecode(snapshot.data!.body);
                final String gifUrl = data['sprites']['versions']
                    ['generation-v']['black-white']['animated']
                    ['front_default'] ??
                    data['sprites']['front_default'];
                final String type = data['types'][0]['type']['name'];
                final Map<String, Color> typeColors = {
                    'fire': Colors.red,
                    'water': Colors.blue,
                    'grass': Colors.green,
                    'electric': Colors.yellow,
                    'bug': Colors.lightGreen,
                    'normal': Colors.grey,
                    'poison': Colors.purple,
                    'ground': Colors.brown,

```

```

    'fairy': Colors.pink,
    'fighting': Colors.orange,
    'psychic': Colors.pinkAccent,
    'rock': Colors.brown,
    'ghost': Colors.deepPurple,
    'ice': Colors.lightBlue,
    'dragon': Colors.indigo,
    'dark': Colors.black,
    'steel': Colors.blueGrey,
    'flying': Colors.indigoAccent,
  };
  return GestureDetector(
    onTap: () {
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (context) => PokemonDetailScreen(
            url: filteredPokemonList[index]['url'],
            typeColor: typeColors[type] ?? Colors.grey),
        ),
      );
    },
    child: Card(
      color: typeColors[type] ?? Colors.grey,
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Image.network(gifUrl, height: 80),
          Text(
            filteredPokemonList[index]['name'].toUpperCase(),
            style: const TextStyle(
              fontWeight: FontWeight.bold,
              color: Colors.white),
          ),
        ],
      ),
    ),
  ],

```

```

        ),
      ),
    );
  },
);
},
),
);
}
}
}

```

```

class PokemonDetailScreen extends StatelessWidget {
  final String url;
  final Color typeColor;

```

```

  const PokemonDetailScreen(
    {super.key, required this.url, required this.typeColor});

```

```

  Future<Map<String, dynamic>> fetchPokemonDetails() async {
    final response = await http.get(Uri.parse(url));
    if (response.statusCode == 200) {
      return jsonDecode(response.body);
    }
    throw Exception('Erro ao carregar dados');
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      backgroundColor: typeColor,
      leading: IconButton(
        icon: const Icon(Icons.arrow_back),
        onPressed: () {
          Navigator.pop(context);
        }
      )
    )
  );
}

```

```

    },
  ),
),
body: FutureBuilder(
  future: fetchPokemonDetails(),
  builder: (context, snapshot) {
    if (!snapshot.hasData) {
      return const Center(child: CircularProgressIndicator());
    }
    final pokemonData = snapshot.data!;
    final String gifUrl = pokemonData['sprites']['versions']
      ['generation-v']['black-white']['animated']
      ['front_default'] ??
      pokemonData['sprites']['front_default'];
    return Container(
      color: typeColor,
      child: Center(
        child: SingleChildScrollView(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              SizedBox(
                height: 150,
                width: 150,
                child: Image.network(
                  gifUrl,
                  fit: BoxFit.contain,
                ),
              ),
              Text(
                pokemonData['name'].toUpperCase(),
                style: const TextStyle(
                  fontSize: 28,
                  fontWeight: FontWeight.bold,
                  color: Colors.white),
              ),
            ],
          ),
        ),
      ),
    ),
  ),
),

```



```

    ),
    Text(
      'Altura: ${((pokemonData['height'] / 10).toStringAsFixed(1))}
m',
      style:
        const TextStyle(fontSize: 18, color: Colors.white)),
    Text(
      'Peso: ${((pokemonData['weight'] / 10).toStringAsFixed(1))}
kg',
      style:
        const TextStyle(fontSize: 18, color: Colors.white)),
    const SizedBox(height: 10),
    const Text('Tipos:',
      style: TextStyle(
        fontSize: 20,
        fontWeight: FontWeight.bold,
        color: Colors.white)),
    Wrap(
      spacing: 10,
      children: (pokemonData['types'] as List)
        .map((type) => Chip(
          label: Text(type['type']['name'].toUpperCase()))
        .toList(),
    ),
    const SizedBox(height: 10),
    const Text('Habilidades:',
      style: TextStyle(
        fontSize: 20,
        fontWeight: FontWeight.bold,
        color: Colors.white)),
    Wrap(
      spacing: 10,
      children: (pokemonData['abilities'] as List)
        .map((ability) => Chip(
          label: Text(

```

```
        ability['ability']['name'].toUpperCase()))
    .toList(),
    ),
    ],
    ),
    ),
    ),
    );
    },
    ),
    );
}
```